Practical Data & BI for Finance

Practical, Hands-On, Job-Ready — India-first + Global

Real formulas, queries, entries & tools — closing the gap between what an MBA teaches and what finance employers pay for

Contents

Why Finance Became a Data Job

What it is & where it's used

Ten years ago a "finance job" meant Excel, a general ledger, and a printer. Today the ledger is a database, the reports are dashboards refreshed hourly, and the person who can *query the raw data* gets promoted over the person who waits for someone to email them a file. This book is about closing that gap.

The shift is structural. Businesses now generate transaction data at a volume Excel physically cannot hold (Excel caps at 1,048,576 rows — a mid-size D2C brand blows past that in a quarter of order lines). So the data moved into databases (SQL Server, PostgreSQL, Snowflake), into ERPs (SAP, Oracle NetSuite, Zoho Books, TallyPrime), and into GST/Income-Tax portals that emit JSON and CSV, not paper. Finance work is now: **pull the data → clean it → model it → explain it.** That pipeline runs on four tools:

Layer	Tool	Finance use
Grab & shape small data	Excel / Google Sheets	Ad-hoc analysis, models, reconciliations
Pull large data from the source	SQL	GL extracts, sales ledgers, GSTR-2B matching
Automate & compute	Python (pandas)	Bank recos, GST reconciliation, forecasting
Present to a decision-maker	Power BI / Looker (DAX)	MIS dashboards, board packs, KPI tracking

Where it's used, by role:

- **FP&A / MIS analyst** — SQL + Excel + Power BI. Builds the monthly MIS.
- **Accounts / R2R (Record-to-Report)** — Tally/SAP + Excel + increasingly SQL for reconciliations.
- **Tax / GST associate** — GST portal + Excel + Python for 2B-vs-books matching at scale.
- **Audit / assurance** — SQL + Excel (CAATs — Computer-Assisted Audit Techniques) to test 100% of transactions, not a sample.
- **Finance business partner / analyst at a startup** — all four, because there's no one else to do it.

The gap: why companies want this (and college didn't teach it)

An MBA Finance or CA Inter curriculum teaches you *accounting standards, valuation, ratio analysis, cost sheets, and tax law.* All essential. None of it teaches you how to get 400,000 sales rows out of a database and reconcile them against GSTR-1 in twenty minutes.

The gap is not knowledge — it's **execution on real, messy, large data.** College gives you a clean 30-row balance sheet in a question paper. The job gives you a 90-column CSV export with blank cells, dates as text, ₹ symbols glued to numbers, and three spellings of the same vendor.

What college taught	What the job actually needs
Prepare a Trial Balance from given data	*Extract* the TB from SAP via SQL, then tie it to sub-ledgers
Calculate current ratio	Build a live ratio dashboard that refreshes every morning
Journal entries by hand	Book 5,000 entries via an import template + validate them
GST computation for one invoice	Reconcile GSTR-2B vs purchase register for 8,000 invoices
VLOOKUP (if at all)	XLOOKUP, Power Query, pivot models, SQL joins

Employers pay a premium for this because it's *rare among finance graduates and expensive to hire separately.* A data analyst who doesn't understand debits/credits can't do it either. You — finance-literate **and** data-capable — are the expensive hybrid.

What "proficient" looks like

The concrete bar. A job-ready person can, **unaided**:

1. Take a raw ERP/bank/GST export and turn it into a clean, analysis-ready table (dedupe, fix types, handle blanks) without manual cell-by-cell editing.
2. Write a `SELECT` with a `JOIN`, `WHERE`, `GROUP BY` and `HAVING` to answer a business question directly from the database.
3. Build an Excel model that uses `XLOOKUP` / `SUMIFS` / `INDEX-MATCH` and doesn't break when a row is inserted.
4. Reconcile two data sets (books vs bank, books vs 2B) and produce a clean *difference* report with reasons.
5. Build a Power BI dashboard with a proper date table and 3–4 DAX measures that a CFO can read without asking questions.
6. Explain, in one sentence, what the number *means* for the business — the finance judgment that a pure data analyst lacks.

If you can do these six, you clear 80% of finance-analyst screens in India today.

Hands-on: how to actually do it

Excel — the modern lookup (stop using VLOOKUP):

```
=XLOOKUP(A2, Vendors[GSTIN], Vendors[VendorName], "Not Found")
```

Sum sales for one region and month:

```
=SUMIFS(Sales[Amount], Sales[Region], "South", Sales[Month], "Jun")
```

Strip a ₹ symbol and text out of an imported number:

```
=VALUE(SUBSTITUTE(SUBSTITUTE(A2,"₹",""),",",""))
```

SQL — pull a revenue summary straight from the ledger:

```
SELECT  c.region,
        SUM(s.amount)          AS total_sales,
        COUNT(*)               AS invoice_count
FROM    sales      s
JOIN    customers  c ON c.customer_id = s.customer_id
WHERE   s.invoice_date ≥ '2026-04-01'
  AND   s.invoice_date <  '2026-07-01'   -- Q1 FY26-27
GROUP BY c.region
HAVING  SUM(s.amount) > 100000
ORDER BY total_sales DESC;
```

Python (pandas) — a GSTR-2B vs purchase-register reconciliation:

```
import pandas as pd

books = pd.read_excel("purchase_register.xlsx")
gstr2b = pd.read_excel("gstr2b.xlsx")

recon = books.merge(
    gstr2b, on=["gstin", "invoice_no"],
    how="outer", suffixes=("_books", "_2b"), indicator=True
)
recon["tax_diff"] = recon["igst_books"].fillna(0) - recon["igst_2b"].fillna(0)

# invoices in books but not in 2B (ITC at risk)
missing_in_2b = recon[recon["_merge"] == "left_only"]
missing_in_2b.to_excel("itc_at_risk.xlsx", index=False)
```

DAX — Power BI measures for an MIS dashboard:

```
Total Sales = SUM(Sales[Amount])

Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])

Sales MoM % =
VAR ThisM = [Total Sales]
VAR PrevM = CALCULATE([Total Sales], DATEADD('Date'[Date], -1, MONTH))
RETURN DIVIDE(ThisM - PrevM, PrevM)
```

TallyPrime — export data you can actually analyse: `Gateway of Tally → Display More Reports → Trial Balance → Alt+E (Export) → File Format: Excel (Spreadsheet) → Enter.` For raw vouchers: `Display More Reports → Day Book → F2 (change period) → Alt+E → Excel.` That export is your `SELECT * FROM ledger` when you have no database access.

A journal entry, the thing the data ultimately represents — credit sale of ₹1,00,000 + 18% GST to a Karnataka customer (intra-state):

Particulars	Dr (₹)	Cr (₹)
Debtors A/c	1,18,000	
To Sales A/c		1,00,000
To Output CGST A/c (9%)		9,000
To Output SGST A/c (9%)		9,000

Worked example / mini-project

Build a one-page monthly MIS from raw exports. Reproduce this with dummy data.

Data: export `Day Book` from Tally to `sales.xlsx` (columns: date, customer, region, amount, gst) — say 4,000 rows for June 2026.

Step 1 — clean in Power Query (Excel/Power BI): `Data → Get Data → From File → Excel`. In the editor: set `date` to Date type, `amount` to Decimal, `Remove Duplicates` on invoice number, `Replace Values` to strip ₹. Close & Load.

Step 2 — the numbers (Excel pivot or SQL):

```
SELECT region,
       SUM(amount)                  AS revenue,
       SUM(amount)*0.18             AS gst_liability,
       ROUND(AVG(amount),0)         AS avg_invoice
FROM   sales
GROUP BY region;
```

Suppose the result:

Region	Revenue (₹)	GST (₹)	Avg invoice (₹)
South	42,00,000	7,56,000	21,000
West	31,50,000	5,67,000	18,500
North	18,00,000	3,24,000	15,000
Total	**91,50,000**	**16,47,000**	—

Step 3 — the insight (the finance bit): South is 46% of revenue but has the highest average invoice — concentration risk if that one region dips. That sentence is what gets you hired; the pivot table is just plumbing.

Step 4 — dashboard: load into Power BI, add the DAX `Total Sales`, `Sales MoM %`, a region bar chart and a KPI card. One page, refreshable. You now have an MIS that took an afternoon and refreshes in one click next month.

How it's tested

Interviews for finance-analyst roles now split into a *talk* round and a *do* round.

Interview questions you'll hear: - "What's the difference between `WHERE` and `HAVING`?" (WHERE filters rows before grouping; HAVING filters after.) - "VLOOKUP vs XLOOKUP — why switch?" (XLOOKUP searches right-to-left, defaults to exact match, returns a not-found value.) - "How would you reconcile GSTR-2B with the purchase register for 8,000 invoices?" - "Walk me through building an MIS from a raw ERP dump."

The practical/assessment tests (the real filter): - **Timed Excel test (30–45 min):** given a messy sheet, build a summary with SUMIFS/XLOOKUP + a pivot. No internet. - **SQL screen (HackerRank/DataCamp or live):** write 3–4 queries with joins and aggregation against a sample schema. - **A "close these books" / reconciliation case:** here are two files, find and explain the differences. - **Case + dashboard take-home:** "Here's 12 months of sales, build a dashboard and tell us what you'd flag to the CFO."

Common mistakes & how pros avoid them

Mistake	What pros do
Manually editing cells to "fix" data	Use Power Query / pandas — repeatable, auditable, one click next month
VLOOKUP everywhere; breaks on column insert	XLOOKUP or INDEX-MATCH; reference by column name
Reconciling by eyeballing two sheets	`merge(..., indicator=True)` or full-outer join; let the machine find diffs
Dumping a 40-column table on the CFO	One page, 3 KPIs, one insight sentence
`SELECT *` on a 10M-row table	Select only needed columns; filter dates in `WHERE`
Numbers with no "so what"	Always end with the business implication
Hard-coding dates/rates in formulas	Parameter cells / a Date table; change once

Learn-it roadmap & resources

Realistic time-to-proficiency for someone with a finance base, ~1 hour/day:

Skill	To job-ready	Free resource	Paid / cert
Modern Excel	3–4 weeks	ExcelJet, Chandoo.org	Microsoft MO-201
SQL	4–6 weeks	SQLBolt, Mode SQL Tutorial, LeetCode DB	DataCamp track
Power BI	3–4 weeks	Microsoft Learn (free), Guy in a Cube (YouTube)	Microsoft PL-300
Python/pandas	6–8 weeks	Kaggle "pandas", freeCodeCamp	DataCamp / Coursera

Sequence: Excel → SQL → Power BI → Python. Excel earns immediately; SQL is the highest ROI hire-signal; Power BI makes you visible to management; Python scales you. Don't chase all four at once — get *proficient* in Excel + SQL first (8–10 weeks) and you're already employable.

Cert worth having in India: Microsoft **PL-300 (Power BI Data Analyst)** — cheap, recognised, and directly maps to MIS roles. SQL needs no cert; a portfolio of 5 solved case dashboards on GitHub beats any certificate.

Quick-reference

```
-- SQL skeleton
SELECT col, SUM(x) FROM t JOIN u ON t.id=u.id
WHERE date ≥ '2026-04-01' GROUP BY col HAVING SUM(x)>0 ORDER BY 2 DESC;
```

```
# pandas reconciliation
a.merge(b, on="key", how="outer", indicator=True)   # _merge: left_only/right_only/both
```

Need	Formula / step
Lookup	`=XLOOKUP(val, lookup_col, return_col, "NA")`
Conditional sum	`=SUMIFS(sum, crit_rng1, c1, crit_rng2, c2)`
Clean ₹ number	`=VALUE(SUBSTITUTE(SUBSTITUTE(A2,"₹",""),",",""))`
Import & clean	`Data → Get Data → Power Query → set types → Close&Load`
Tally export	`Report → Alt+E → Excel`
DAX YoY	`TOTALYTD([Measure], 'Date'[Date])`
GST (intra-state)	Output CGST 9% + SGST 9% on taxable value
Excel row limit	1,048,576 — past this, use SQL

The one idea: finance is now *pull → clean → model → explain*. Master the pipeline, keep the judgment, and you're the hybrid employers overpay for.

SQL for Finance: Foundations

What it is & where it's used

SQL (Structured Query Language) is how you *ask questions of data that lives in a database* instead of a spreadsheet. When your company's transactions, ledgers, invoices, and customer masters outgrow Excel — usually past a few hundred thousand rows — they move to a database (PostgreSQL, MySQL, SQL Server, Snowflake, or the SQL layer inside your ERP). SQL is the language you use to pull, filter, and summarise that data.

In finance and accounting, SQL is the everyday tool for:

- **FP&A / financial analysts** — pulling actuals from the GL to build variance reports and MIS.
- **Accounts / R2R teams** — reconciling sub-ledgers to control accounts, ageing analysis, finding unposted entries.
- **Tax / GST analysts** — extracting invoice-level data to reconcile GSTR-1 vs books, matching ITC.
- **Internal audit & controls** — sampling transactions, testing for duplicate payments, journal-entry testing.
- **Revenue / collections** — customer ageing, DSO, overdue exposure.

You do not need to *build* databases. You need to *read* from them confidently. That is 90% of the finance SQL job, and it rests on six clauses: `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `HAVING`, `ORDER BY`.

The gap: why companies want this (and college didn't teach it)

An MBA or CA course teaches you *what* a trial balance, an ageing schedule, or a GST reconciliation should look like. It assumes the data arrives clean. In a real company, the data sits in a database with 40 tables, cryptic column names, and 2 million rows — and *nobody hands you the summarised sheet*. You have to extract it yourself.

The specific gap:

- College says "the accountant will give you the ledger." Industry says "here's read access to the `gl_transactions` table — get what you need."
- Excel breaks, freezes, or silently truncates past ~1M rows. A single month of a mid-size company's transactions can exceed that.
- Analysts who can't write SQL become *dependent on the IT/BI team* for every data request, waiting days for a pull. Analysts who can write SQL are self-sufficient — and that self-sufficiency is exactly what gets you hired and promoted.

Employers have stopped treating SQL as an "IT skill." For any finance analyst role in a mid-to-large company, a basic SQL screen is now as standard as an Excel test.

What "proficient" looks like

The bar for a finance analyst (not a data engineer) is narrow and achievable. Job-ready means you can, *unaided*:

- Write a `SELECT` with a `WHERE` filter on dates, amounts, and text — and get the exact rows you intend.

- `JOIN` two or three tables (e.g. transactions → customers → GL accounts) and know the difference between `INNER` and `LEFT JOIN`.
- Aggregate with `GROUP BY` (sum by account, by month, by customer) and filter groups with `HAVING`.
- Sort and rank with `ORDER BY ... DESC` and `LIMIT`.
- Read a query someone else wrote and explain what number it produces.
- Sanity-check your output against a known control total (does the SUM tie to the trial balance?).

You do *not* need window functions, stored procedures, or query tuning to clear this bar — those come later.

Hands-on: how to actually do it

Assume three tables. This is a realistic mini-schema for an Indian company.

`gl_transactions`

txn_id	txn_date	account_code	customer_id	debit	credit	narration
1001	2026-04-05	4000	C-01	0	118000	Sales invoice INV-201
1002	2026-04-05	1200	C-01	118000	0	Debtor INV-201
1003	2026-04-08	5100	NULL	45000	0	Office rent

`accounts` (`account_code`, `account_name`, `account_type`) — the chart of accounts. **`customers`** (`customer_id`, `customer_name`, `state`, `gstin`).

SELECT + WHERE — pick columns and filter rows

```
-- All sales-account entries in April 2026, above ₹50,000
SELECT txn_date, account_code, credit, narration
FROM   gl_transactions
WHERE  account_code = 4000
  AND  txn_date BETWEEN '2026-04-01' AND '2026-04-30'
  AND  credit > 50000;
```

Key operators: `=`, `<>`, `>`, `<`, `BETWEEN`, `IN (…)`, `LIKE '%rent%'` (text search), `IS NULL`. Combine with `AND` / `OR`. Wrap `OR` groups in brackets so precedence is unambiguous.

JOIN — combine tables

```
-- Attach account names and types to each transaction
SELECT t.txn_date, a.account_name, a.account_type,
       t.debit, t.credit
FROM   gl_transactions t
JOIN   accounts a  ON t.account_code = a.account_code
WHERE  t.txn_date ≥ '2026-04-01';
```

- **`INNER JOIN`** (just `JOIN`) — keeps only rows that match in both tables.

- `LEFT JOIN` — keeps *all* left-table rows; unmatched right side comes back `NULL`. Use `LEFT JOIN` when you want to find what's *missing*:

```
-- Customers with NO transactions this year (dormant accounts)
SELECT c.customer_id, c.customer_name
FROM   customers c
LEFT JOIN gl_transactions t ON c.customer_id = t.customer_id
WHERE  t.txn_id IS NULL;
```

GROUP BY + HAVING — summarise, then filter groups

```
-- Net movement per GL account, only accounts with net > ₹1,00,000
SELECT a.account_name,
       SUM(t.debit)  AS total_debit,
       SUM(t.credit) AS total_credit,
       SUM(t.debit) - SUM(t.credit) AS net_debit
FROM   gl_transactions t
JOIN   accounts a ON t.account_code = a.account_code
GROUP BY a.account_name
HAVING SUM(t.debit) - SUM(t.credit) > 100000
ORDER BY net_debit DESC;
```

The mental model: `WHERE` filters **rows** *before* grouping; `HAVING` filters **groups** *after* aggregation. You cannot put `SUM()` in a `WHERE` — that is what `HAVING` is for.

ORDER BY — sort the result

```
ORDER BY net_debit DESC   -- largest first
ORDER BY txn_date ASC, txn_id ASC   -- oldest first, tie-break by id
```

Add `LIMIT 10` (MySQL/Postgres) or `TOP 10` (SQL Server) to get the top N.

Aggregate functions you'll use daily

`SUM()`, `COUNT(*)`, `COUNT(DISTINCT customer_id)`, `AVG()`, `MIN()`, `MAX()`. Wrap monthly grouping with a date-truncation function — dialect varies: Postgres `DATE_TRUNC('month', txn_date)`, MySQL `DATE_FORMAT(txn_date,'%Y-%m')`, SQL Server `FORMAT(txn_date,'yyyy-MM')`.

Worked example / mini-project

Goal: a debtor ageing summary by state for the quarter — the kind of MIS a finance analyst produces monthly.

Setup (reproduce in any free SQL sandbox — SQLite via sqliteonline.com needs no install):

```
CREATE TABLE invoices (
  inv_id INTEGER, customer_id TEXT, inv_date TEXT,
  due_date TEXT, amount REAL, status TEXT
);
INSERT INTO invoices VALUES
 (201,'C-01','2026-04-05','2026-05-05',118000,'OPEN'),
 (202,'C-02','2026-04-18','2026-05-18',236000,'OPEN'),
 (203,'C-01','2026-03-02','2026-04-01', 59000,'OPEN'),
 (204,'C-03','2026-05-10','2026-06-09',472000,'PAID'),
 (205,'C-02','2026-02-15','2026-03-17', 90000,'OPEN');

CREATE TABLE customers (customer_id TEXT, customer_name TEXT, state TEXT);
INSERT INTO customers VALUES
 ('C-01','Sharma Traders','Karnataka'),
 ('C-02','Nova Textiles','Maharashtra'),
 ('C-03','Delta Foods','Karnataka');
```

Now the ageing query. Assume "today" is 2026-06-30.

```
SELECT c.state,
       COUNT(*)              AS open_invoices,
       SUM(i.amount)         AS total_outstanding,
       SUM(CASE WHEN julianday('2026-06-30') - julianday(i.due_date) ≤ 30
                THEN i.amount ELSE 0 END) AS due_0_30,
       SUM(CASE WHEN julianday('2026-06-30') - julianday(i.due_date) > 30
                THEN i.amount ELSE 0 END) AS overdue_30_plus
FROM   invoices i
JOIN   customers c ON i.customer_id = c.customer_id
WHERE  i.status = 'OPEN'
GROUP BY c.state
ORDER BY total_outstanding DESC;
```

Result:

state	open_invoices	total_outstanding	due_0_30	overdue_30_plus
Maharashtra	2	326000	236000	90000
Karnataka	2	177000	0	177000

In four clauses you filtered to open invoices (`WHERE`), attached the state (`JOIN`), bucketed the amounts (`CASE` inside `SUM`), rolled up by state (`GROUP BY`), and ranked by exposure (`ORDER BY`). That is a real deliverable, not a toy.

How it's tested

Companies test SQL two ways.

1. The live SQL screen (30–45 min, shared editor or HackerRank). You get a schema and are asked to write queries against it. Typical prompts:

- "Return total sales per customer for FY26, highest first." (`JOIN` + `GROUP BY` + `ORDER BY`)
- "Find customers with more than 5 transactions but total spend under ₹10,000." (`GROUP BY` + `HAVING`)
- "List invoices with no matching payment." (`LEFT JOIN … IS NULL`)
- "What's the difference between `INNER` and `LEFT JOIN` ? When would net totals differ?"
- " `WHERE` vs `HAVING` — why can't I filter `SUM()` in the `WHERE` ?"

2. The take-home / case. A CSV or database dump plus a business question: "Reconcile this sub-ledger to the GL and list the mismatched accounts," or "Produce a monthly revenue trend." They're checking whether your number *ties* to a control total — accuracy over cleverness.

Interview verbal questions also probe: "What does `COUNT(*)` count vs `COUNT(column)` ?" (the latter skips `NULL` s), and "Why did your `JOIN` inflate the total?" (a one-to-many join fanning out rows — a classic trap).

Common mistakes & how pros avoid them

Mistake	Symptom	Fix
Filtering an aggregate in `WHERE`	`SQL error: aggregate not allowed`	Move it to `HAVING` .
Double-counting from a one-to-many `JOIN`	SUM is too high, won't tie to TB	Aggregate first in a subquery, then join; or `COUNT(DISTINCT …)` .
`INNER JOIN` silently dropping rows	Total is *lower* than expected	Use `LEFT JOIN` when the right table may not match; then check for `NULL` .
Forgetting `NULL` logic	Rows with `NULL` vanish from `<>` filters	`NULL` is not equal to anything; use `IS NULL` / `COALESCE(col,0)` .
Ambiguous date strings	Wrong month pulled	Always `YYYY-MM-DD` ; use half-open ranges `≥ '2026-04-01' AND < '2026-05-01'` .
Non-aggregated column in `GROUP BY`	Error or wrong grouping	Every non-aggregated `SELECT` column must appear in `GROUP BY` .
Not sanity-checking	Wrong number ships to management	Always tie your `SUM` to a known control (trial balance, invoice register).

Pros write the `WHERE` filter *first* and eyeball a few raw rows before they aggregate — you cannot trust a total you haven't spot-checked.

Learn-it roadmap & resources

Realistic time to the finance-analyst bar (able to clear a SQL screen): **3–5 weeks at ~1 hour/day.** SQL rewards small daily reps on a real dataset far more than long theory sessions.

Week	Focus
1	`SELECT`, `WHERE`, operators, `ORDER BY`, `LIMIT`
2	`JOIN` (inner/left), multi-table queries
3	`GROUP BY`, aggregates, `HAVING`, `CASE`
4	Subqueries, reconciliation-style problems, date handling
5	Timed practice on HackerRank/StrataScratch

Free: SQLBolt (interactive, ~2 hrs to core), Mode Analytics SQL Tutorial (has an *Analytics* track with real business data), Kaggle "Intro to SQL", W3Schools (reference), sqliteonline.com (zero-install practice sandbox).
Practice with pressure: HackerRank SQL track (what many screens are built on), LeetCode Database, StrataScratch (real finance/company questions).

Paid / certification (optional, only if a JD asks): Microsoft **PL-300 (Power BI Data Analyst)** includes SQL-adjacent skills; **Google Data Analytics Certificate** (Coursera) covers SQL for analysts. For finance roles in India, a certificate is rarely required — a clean take-home submission carries more weight.

Quick-reference

```
SELECT col1, SUM(col2) AS total   -- pick columns / aggregate
FROM   table_a a
JOIN   table_b b ON a.key = b.key -- INNER: matches only
LEFT JOIN table_c c ON …          -- keep all left rows
WHERE  a.date ≥ '2026-04-01'      -- filter ROWS (before grouping)
  AND  a.amount > 50000
GROUP BY col1                     -- one output row per group
HAVING SUM(col2) > 100000         -- filter GROUPS (after aggregation)
ORDER BY total DESC               -- sort; ASC is default
LIMIT 10;                         -- top N (TOP 10 in SQL Server)
```

Need	Clause / function
Filter rows	`WHERE`
Filter aggregated groups	`HAVING`
Text search	`LIKE '%text%'`
Match a list	`IN ('A','B')`
Range	`BETWEEN x AND y`
Missing values	`IS NULL` , `COALESCE(col,0)`
Count uniques	`COUNT(DISTINCT col)`
Bucketing (ageing)	`CASE WHEN … THEN … ELSE … END`
Find unmatched rows	`LEFT JOIN … WHERE right.key IS NULL`
Month grouping	`DATE_TRUNC` / `DATE_FORMAT` / `FORMAT`

Order of clauses (always): `SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT` .
Order of execution (what actually runs): `FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT` . Knowing execution order explains why you can't reference a `SELECT` alias in `WHERE` , and why `HAVING` sees aggregates that `WHERE` can't.

SQL for Finance: Practical Queries

What it is & where it's used

SQL (Structured Query Language) is how you ask a database questions. In finance, the "database" is where the real numbers live — the general ledger, the sales/AR sub-ledger, the payments table, the customer master — usually in PostgreSQL, MySQL, SQL Server, Snowflake, or BigQuery. Excel breaks past ~1 million rows and gets slow at 100k; SQL joins 50 million rows of transactions to a customer table in seconds and never silently corrupts a VLOOKUP.

Roles that live in SQL every day:

Role	What they query
FP&A / Finance analyst	Revenue trends, budget vs actual, cohort retention
Revenue / Billing analyst	MRR/ARR, churn, invoicing accuracy
Accounts / AR-AP analyst	Ageing buckets, unapplied cash, reconciliations
Internal audit / controls	Duplicate payments, journal anomalies, three-way match
Data/BI analyst supporting finance	The pipelines feeding Power BI/Tableau

In Indian startups, fintechs, and GCCs (Global Capability Centres in Bengaluru/Hyderabad/Pune), "SQL is required" appears on almost every finance-analyst JD above fresher level. It is the single highest-leverage non-accounting skill a CA-track candidate can add.

The gap: why companies want this (and college didn't teach it)

An MBA Finance / CA syllabus teaches you *what* a revenue figure means, how to age receivables, and the accounting behind a reconciliation. It never shows you that in a real company those numbers sit in a `transactions` table with 8 million rows and you are expected to produce them yourself, unaided, by lunchtime.

The specific gap: - **You know the concept of ageing; you've never written the query that buckets 40,000 open invoices.** College gave you a 10-row textbook example. Work gives you a raw table and a deadline. - **You've reconciled two statements by hand in Excel.** Nobody told you `FULL OUTER JOIN` does the same match for a million rows and flags every break automatically. - **You think of data as a finished report.** Employers think of it as rows you filter, join, and aggregate on demand.

Closing this gap means you stop asking the data team for extracts and start answering your own questions — which is exactly what makes an analyst "senior."

What "proficient" looks like

The bar employers actually test:

1. **You can `JOIN` two or three tables** (invoices to customers to payments) without producing duplicate rows, and you know *why* an inner vs left join changes the answer.

2. **You use `GROUP BY` with `HAVING`** to aggregate and filter aggregates (e.g. customers with total billing > ₹10 lakh).
3. **You write window functions** — `SUM() OVER`, `RANK()`, `LAG()` — for running totals, rankings, and month-on-month growth.
4. **You structure a multi-step query with CTEs** (`WITH`) instead of nesting subqueries five deep.
5. **You handle dates** — truncate to month, compute ageing days, filter a fiscal year.
6. **You reconcile** two sources with a `FULL OUTER JOIN` and isolate the breaks.

If you can do those six unaided on an unfamiliar schema, you pass 90% of finance SQL screens.

Hands-on: how to actually do it

Assume this schema (typical AR/revenue setup):

```
customers(customer_id, customer_name, segment, region)
invoices(invoice_id, customer_id, invoice_date, due_date, amount, status)
payments(payment_id, invoice_id, payment_date, amount_paid)
```

Filtering and aggregating — revenue by month:

```
SELECT DATE_TRUNC('month', invoice_date) AS month,
       SUM(amount)                       AS revenue,
       COUNT(*)                          AS num_invoices
FROM   invoices
WHERE  status <> 'cancelled'
GROUP  BY DATE_TRUNC('month', invoice_date)
ORDER  BY month;
```

Joins — revenue by region (invoices + customers):

```
SELECT c.region,
       SUM(i.amount) AS revenue
FROM   invoices i
JOIN   customers c ON c.customer_id = i.customer_id
GROUP  BY c.region
ORDER  BY revenue DESC;
```

Window function — running (cumulative) total of revenue by month:

```
SELECT month,
       revenue,
       SUM(revenue) OVER (ORDER BY month
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
                    AS running_total
FROM (
    SELECT DATE_TRUNC('month', invoice_date) AS month,
           SUM(amount)                       AS revenue
    FROM   invoices
    GROUP  BY 1
) m
ORDER BY month;
```

Window function — rank top customers, and month-on-month growth with `LAG`:

```
-- Top 5 customers by billing
SELECT customer_id,
       SUM(amount) AS total_billed,
       RANK() OVER (ORDER BY SUM(amount) DESC) AS rnk
FROM   invoices
GROUP  BY customer_id
QUALIFY rnk ≤ 5;          -- Snowflake/BigQuery; else wrap in a subquery
```

```
-- Month-on-month % growth
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / LAG(revenue) OVER (ORDER BY month), 1) AS growth_pct
FROM   monthly_rev;
```

CTEs — chaining steps readably:

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', invoice_date) AS month,
           SUM(amount) AS revenue
    FROM   invoices
    GROUP  BY 1
),
with_growth AS (
    SELECT month, revenue,
           revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
    FROM   monthly
)
SELECT * FROM with_growth WHERE mom_change < 0;   -- months that declined
```

Date logic — AR ageing buckets:

```
SELECT i.invoice_id, c.customer_name, i.amount,
       CURRENT_DATE - i.due_date AS days_overdue,
       CASE
         WHEN CURRENT_DATE - i.due_date ≤ 0   THEN 'Not due'
         WHEN CURRENT_DATE - i.due_date ≤ 30  THEN '0-30'
         WHEN CURRENT_DATE - i.due_date ≤ 60  THEN '31-60'
         WHEN CURRENT_DATE - i.due_date ≤ 90  THEN '61-90'
         ELSE '90+'
       END AS bucket
FROM   invoices i
JOIN   customers c ON c.customer_id = i.customer_id
WHERE  i.status = 'open';
```

Reconciliation — match GL to bank/sub-ledger and find breaks:

```
SELECT COALESCE(g.txn_id, b.txn_id) AS txn_id,
       g.amount AS gl_amount,
       b.amount AS bank_amount,
       CASE
         WHEN b.txn_id IS NULL THEN 'Missing in bank'
         WHEN g.txn_id IS NULL THEN 'Missing in GL'
         WHEN g.amount <> b.amount THEN 'Amount mismatch'
       END AS break_type
FROM   gl g
FULL OUTER JOIN bank b ON g.txn_id = b.txn_id
WHERE  g.txn_id IS NULL
   OR  b.txn_id IS NULL
   OR  g.amount <> b.amount;
```

Worked example / mini-project

Scenario: You're the FP&A analyst at an Indian SaaS company. Build a monthly-cohort retention view — do customers acquired in a given month keep paying?

Sample `invoices` data (₹, one row per customer-month):

customer_id	invoice_date	amount
1	2025-01-15	5,000
1	2025-02-15	5,000
1	2025-03-15	5,000
2	2025-01-20	8,000
2	2025-02-20	8,000
3	2025-02-10	3,000

Step 1 — find each customer's cohort (first billing month):

```
WITH cohort AS (
    SELECT customer_id,
           MIN(DATE_TRUNC('month', invoice_date)) AS cohort_month
    FROM   invoices
    GROUP  BY customer_id
),
activity AS (
    SELECT i.customer_id,
           c.cohort_month,
           DATE_TRUNC('month', i.invoice_date) AS active_month
    FROM   invoices i
    JOIN   cohort c ON c.customer_id = i.customer_id
)
SELECT cohort_month,
       -- months since acquisition (0, 1, 2 ...)
       (EXTRACT(YEAR FROM active_month) - EXTRACT(YEAR FROM cohort_month)) * 12
        + (EXTRACT(MONTH FROM active_month) - EXTRACT(MONTH FROM cohort_month)) AS month_offse
       COUNT(DISTINCT customer_id) AS active_customers,
       SUM(1) AS billings_count
FROM   activity
GROUP  BY cohort_month, month_offset
ORDER  BY cohort_month, month_offset;
```

Result (retention triangle):

cohort_month	month_offset	active_customers
2025-01	0	2
2025-01	1	2
2025-01	2	1
2025-02	0	1

Read it: the Jan cohort started with 2 customers, both stayed in month 1, one dropped by month 2 (50% retention). That single query — cohort CTE + offset date math + `COUNT(DISTINCT)` — is exactly the deliverable a startup finance team calls "the retention curve." Reproduce it locally with any free SQLite/PostgreSQL install and the six rows above.

How it's tested

Companies test SQL far more concretely than accounting:

The live SQL screen (most common). You share screen on HackerRank / DataLemur / a scratch database and solve 3-5 problems in 45 minutes. Typical prompts: - "Return the top 3 customers by revenue per region." (window function `RANK()` partitioned by region) - "Show month-on-month revenue growth %." (`LAG`) - "Bucket these open invoices into ageing bands." (`CASE` + date math) - "Find invoices with no matching payment." (`LEFT JOIN … WHERE payments.id IS NULL`)

Take-home case. "Here's a CSV of 20k transactions — build the AR ageing report and list the 10 largest overdue balances." They check whether your joins double-count and whether your buckets are right at the boundaries.

Conceptual interview questions: - Difference between `WHERE` and `HAVING` ? (row filter vs group filter) - `INNER` vs `LEFT` vs `FULL OUTER JOIN` — when does each change a reconciliation result? - What does a window function do that `GROUP BY` cannot? (keeps every row while adding an aggregate) - Why is `COUNT(*)` different from `COUNT(column)` ? (the latter ignores NULLs)

Common mistakes & how pros avoid them

Mistake	Why it hurts	Pro habit
Joins that fan out rows	Summing a customer's invoices *after* joining to multiple payments doubles revenue	Aggregate each side in a CTE *before* joining, or join on a unique key
Filtering a `LEFT JOIN` in `WHERE`	`WHERE b.col = x` silently turns it back into an inner join and hides the breaks	Put the condition in the `ON`, or filter for `IS NULL`
Ageing boundary errors	`≤ 30` and `≥ 30` both include day 30 → double-count	Use clean `≤ 30` then `≤ 60`, never overlapping ranges
`SELECT *` on a huge table	Pulls 40 columns you don't need, slow and unreadable	Select only the columns you'll use
Ignoring NULLs in reconciliation	A NULL amount silently drops from `SUM` or a `=` comparison	`COALESCE(amount,0)`; test `IS NULL` explicitly
Trusting `GROUP BY` without checking counts	You never notice the join fanned out	Sanity-check: does total match the known control figure?
Rounding currency mid-query	Compounds paise-level errors	Round once, at the final SELECT

Learn-it roadmap & resources

Realistic time-to-proficiency for a finance person who already knows Excel: **6-8 weeks, ~45 min/day.**

Week	Focus
1	SELECT, WHERE, ORDER BY, basic aggregates
2	GROUP BY, HAVING, the join types
3	CTEs (`WITH`), subqueries
4	Window functions — the finance sweet spot
5	Date functions, `CASE`, ageing/cohort patterns
6-8	Timed practice on real finance-style problems

Free: - **SQLBolt** and **Mode SQL Tutorial** — interactive, browser-based, start here. - **DataLemur** (Ambitious with SQL) — window-function and analytics problems, many finance-flavoured. - **PostgreSQL** + **DBeaver**, both free — install locally and load your own CSVs. - **StrataScratch** free tier — real company-style questions.

Paid / certification: - **Google Data Analytics Professional Certificate** (Coursera) — includes SQL, recruiter-recognised in India. - **Microsoft PL-300 / DP-900** if you're pairing SQL with Power BI/Azure. - **HackerRank SQL (Intermediate/Advanced) badge** — free to earn, cheap signal to put on a CV.

For a CA-track candidate, SQL + Power BI is the combination that moves you from "accounts executive" to "finance analyst" pay bands.

Quick-reference

```
-- Month truncation
DATE_TRUNC('month', d)              -- Postgres
DATE_FORMAT(d,'%Y-%m-01')           -- MySQL
FORMAT(d,'yyyy-MM')                 -- SQL Server

-- Days overdue
CURRENT_DATE - due_date             -- Postgres
DATEDIFF(day, due_date, GETDATE())  -- SQL Server
```

Need	Pattern
Running total	`SUM(x) OVER (ORDER BY d ROWS UNBOUNDED PRECEDING)`
Rank	`RANK() OVER (PARTITION BY region ORDER BY sales DESC)`
Prev period	`LAG(x) OVER (ORDER BY month)`
Next period	`LEAD(x) OVER (ORDER BY month)`
Multi-step query	`WITH cte1 AS (...), cte2 AS (...) SELECT ...`
Ageing bucket	`CASE WHEN days≤30 THEN '0-30' ... END`
Unmatched rows	`LEFT JOIN ... WHERE b.id IS NULL`
Full reconciliation	`FULL OUTER JOIN ... WHERE a.id IS NULL OR b.id IS NULL OR a.amt<>b.amt`
Filter aggregates	`GROUP BY ... HAVING SUM(x) > 1000000`
Distinct count	`COUNT(DISTINCT customer_id)`

Order of execution to remember: `FROM → JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → ORDER BY → LIMIT`. Knowing this explains why you can't reference a `SELECT` alias in `WHERE` but can in `ORDER BY`.

Python for Finance I: pandas & numpy

What it is & where it's used

pandas is a Python library that gives you a spreadsheet-like object called a *DataFrame* — rows and columns, but programmable. **numpy** is the numerical engine underneath it (fast arrays and math). Together they are the workhorses of every finance analytics team: you load a CSV/Excel/database extract, clean it, filter it, group it, join it to a master, and produce numbers — all in code that runs identically tomorrow with one click.

Where it shows up in real finance/accounts/tax roles:

Role	What they do with pandas
FP&A / MIS analyst	Combine month-end exports into a P&L variance report
Accounts / AR-AP	Reconcile ledger vs bank, ageing of receivables
Tax / GST associate	Match GSTR-2B (portal) against purchase register (Tally/ERP)
Audit / internal audit	Sample testing, duplicate-invoice detection across lakhs of rows
Equity / credit research	Clean price/fundamental data, compute ratios and returns

Excel breaks past ~100k rows and can't reliably repeat a 20-step manual process. pandas doesn't care whether it's 500 rows or 5 million, and the process is saved as a script.

The gap: why companies want this (and college didn't teach it)

An MBA-Finance or CA syllabus teaches you *what* a receivables ageing or a GST reconciliation **means**. It does not teach you to produce one from a 2-lakh-row dump at 9 PM on close day. Colleges assume "the data is given, clean, and small." Industry data is none of those: it arrives as a messy export with merged headers, ₹ signs stuck in text, dates as `31-03-2026` in one file and `2026/03/31` in another, and duplicate invoice numbers.

The gap is **repeatable data handling at scale**. Employers have discovered that a fresher who can write 15 lines of pandas replaces hours of copy-paste-VLOOKUP and, crucially, does it *the same way every month* with an audit trail. That reliability — not fancy modelling — is what they pay a premium for.

What "proficient" looks like

A job-ready person can, **unaided**, take two raw files and produce a correct output in under 30 minutes:

- Load CSV/Excel (right sheet, skip junk rows, set dtypes).
- Inspect: `.head()`, `.info()`, `.describe()`, `.shape`.
- Filter rows on multiple conditions and select columns.
- Clean: strip whitespace, fix ₹/comma text into numbers, parse dates, handle blanks/NaN, drop or flag duplicates.
- `groupby` + aggregate to get subtotals (sum, count, mean) by any dimension.
- `merge` two DataFrames (inner/left) on a key — and know *why* rows drop or duplicate.

- Add computed columns, sort, and export back to Excel with formatting intact.

If you can do a GST-2B vs purchase-register match or an AR ageing without googling every second line, you clear the bar.

Hands-on: how to actually do it

Setup once: `pip install pandas numpy openpyxl`. Then `import pandas as pd` and `import numpy as np`.

Load data

```
# CSV — set the invoice number as text so leading zeros survive
df = pd.read_csv("sales.csv", dtype={"invoice_no": str})

# Excel — pick sheet, skip 3 junk header rows, parse a date column
df = pd.read_excel("tb.xlsx", sheet_name="Data", skiprows=3,
                   parse_dates=["invoice_date"])
```

Inspect

```
df.shape            # (rows, cols)
df.head(10)         # first 10 rows
df.info()           # dtypes + non-null counts — catches text-vs-number bugs
df.describe()       # min/max/mean of numeric cols
df["state"].value_counts()
```

Filter & select (`&` = and, `|` = or, each condition in parentheses)

```
# Maharashtra invoices above ₹1,00,000
big = df[(df["state"] == "Maharashtra") & (df["amount"] > 100000)]

# only three columns
cols = df[["invoice_no", "party", "amount"]]

# .isin for a list; ~ negates
south = df[df["state"].isin(["Karnataka", "Tamil Nadu", "Kerala"])]
```

Clean — the part that eats 80% of real work

```
# "₹1,20,500" (text) → 120500.0 (number)
df["amount"] = (df["amount"].astype(str)
                .str.replace("₹", "", regex=False)
                .str.replace(",", "", regex=False)
                .str.strip())
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")  # bad → NaN

# trim stray spaces in text keys (a top cause of failed merges)
df["party"] = df["party"].str.strip().str.upper()

# mixed date formats → real dates
df["invoice_date"] = pd.to_datetime(df["invoice_date"],
                                    dayfirst=True, errors="coerce")

df["amount"] = df["amount"].fillna(0)        # blanks → 0
df = df.drop_duplicates(subset=["invoice_no"])  # kill dupes on key

# flag (don't delete) duplicates for review
df["is_dup"] = df.duplicated(subset=["invoice_no"], keep=False)
```

groupby — subtotals

```
# total & count of sales per state
by_state = (df.groupby("state")
              .agg(total=("amount", "sum"),
                   invoices=("invoice_no", "count"))
              .reset_index()
              .sort_values("total", ascending=False))
```

merge — the VLOOKUP replacement

```
merged = pd.merge(purchase, gstr2b,
                  on="invoice_no", how="left",
                  suffixes=("_book", "_portal"),
                  indicator=True)   # _merge col shows both / left_only
```

numpy for conditional columns

```
df["gst_rate"] = np.where(df["amount"] > 50000, 0.18, 0.12)
df["gst"] = df["amount"] * df["gst_rate"]
```

Export

```
merged.to_excel("recon_output.xlsx", index=False)
```

Worked example / mini-project: GST-2B vs Purchase-Register match

You have the **purchase register** from Tally and **GSTR-2B** downloaded from the GST portal. Goal: find invoices where Input Tax Credit (ITC) in your books doesn't match the portal — the classic monthly ITC reconciliation.

`purchase_register.csv`

invoice_no	supplier_gstin	taxable	igst
INV001	27ABCDE1234F1Z5	100000	18000
INV002	29PQRST5678G2Z1	50000	9000
INV003	27ABCDE1234F1Z5	20000	3600

`gstr2b.csv` (portal shows INV001 with a different tax, and no INV003)

invoice_no	igst
INV001	17000
INV002	9000

```
import pandas as pd

pr  = pd.read_csv("purchase_register.csv", dtype={"invoice_no": str})
b2b = pd.read_csv("gstr2b.csv", dtype={"invoice_no": str})

# clean keys so the match doesn't fail on spaces/case
for d in (pr, b2b):
    d["invoice_no"] = d["invoice_no"].str.strip().str.upper()

m = pd.merge(pr, b2b, on="invoice_no", how="left",
             suffixes=("_book", "_2b"), indicator=True)

m["igst_2b"] = m["igst_2b"].fillna(0)
m["diff"] = m["igst_book"] - m["igst_2b"]

def status(r):
    if r["_merge"] == "left_only":
        return "Missing in 2B (ITC at risk)"
    return "OK" if abs(r["diff"]) < 1 else "Tax mismatch"

m["status"] = m.apply(status, axis=1)
print(m[["invoice_no", "igst_book", "igst_2b", "diff", "status"]])
```

Output:

invoice_no	igst_book	igst_2b	diff	status
INV001	18000	17000	1000	Tax mismatch
INV002	9000	9000	0	OK
INV003	3600	0	3600	Missing in 2B (ITC at risk)

You just flagged ₹3,600 of ITC you can't claim yet and a ₹1,000 mismatch to query the supplier — the exact deliverable a GST associate produces every month, now reproducible in seconds.

How it's tested

Interview questions (conceptual): - Difference between `merge` types — what happens to unmatched rows in `inner` vs `left`? - Why did your merge produce *more* rows than either input? (Answer: duplicate keys → many-to-many.) - `loc` vs `iloc`? When does a "SettingWithCopyWarning" appear? - How do you replace a nested VLOOKUP / a pivot table with pandas? - Difference between `apply`, `map`, and vectorised operations (and why vectorised is faster).

Practical test (what actually decides it): a timed take-home or live screen. Typical brief: "*Here are two CSVs — a sales dump and a customer master. Give me total sales per region, flag customers with no master record, and export to Excel. 30 minutes.*" They watch whether you check dtypes, handle the dirty ₹ column, pick the right join, and notice dropped rows. Some firms give a Jupyter notebook and one messy file and simply say "reconcile these."

Common mistakes & how pros avoid them

Mistake	Consequence	Pro habit
Merging on untrimmed/mixed-case keys	Silent non-matches	`.str.strip().str.upper()` both keys first
Ignoring `.info()` — amount is text	`sum()` concatenates or errors	Check dtypes immediately after load
`how="inner"` by default	Silently drops unmatched rows	Use `how="left"` + `indicator=True` to see drops
Not checking row count after merge	Duplicated rows inflate totals	Compare `len()` before/after; check key uniqueness
Chained indexing `df[df.x>1]["y"]=0`	SettingWithCopyWarning, no change	Use `.loc[mask, "y"] = 0`
`dropna()` everything	Deletes valid rows with one blank	Fill or subset: `fillna(0)`, `dropna(subset=[...])`
Losing leading zeros in invoice/GSTIN	Broken keys	`dtype=str` on read

Golden rule: after every merge, print `df["_merge"].value_counts()` and confirm row counts. Most "wrong numbers" in finance pandas are a bad join, not bad math.

Learn-it roadmap & resources

Time to job-ready: 3–5 weeks at ~1 hr/day if you already know finance logic (you do).

Week	Focus
1	Python basics + read/inspect CSV/Excel; `head/info/describe`
2	Filtering, `loc/iloc`, cleaning (dtypes, dates, ₹-text, NaN)
3	`groupby` aggregations; `merge` / `concat`; computed columns
4	Rebuild 2–3 of your own Excel reports in pandas end-to-end
5	Speed, `pivot_table`, export formatting, edge cases

Resources - *pandas official "10 minutes to pandas"* — free, start here. - Kaggle **"Pandas" micro-course** — free, hands-on, ~4 hrs. - Wes McKinney, *Python for Data Analysis* (3rd ed.) — the reference (pandas' creator). - Practice on real data: RBI/NSE downloads, or your own bank statement/Tally exports.

Certification: none is required for pandas itself; recruiters test the skill, not a badge. If you want a credential, *Google Data Analytics (Coursera)* or a Kaggle certificate signals intent. Your strongest asset is a GitHub repo with 2–3 reconciliations like the one above.

Quick-reference

```
import pandas as pd, numpy as np

pd.read_csv("f.csv", dtype={"id": str})          # load, keep id as text
pd.read_excel("f.xlsx", sheet_name="S", skiprows=2, parse_dates=["dt"])
df.head(); df.info(); df.describe(); df.shape     # inspect
df["c"].value_counts()                            # frequency

df[(df.a > 100) & (df.b == "X")]                  # filter (parentheses!)
df[df.state.isin(["KA","TN"])]                    # value in list
df.loc[mask, "col"] = 0                           # safe assignment

pd.to_numeric(s, errors="coerce")                 # text → number
s.str.replace(",", "", regex=False)               # strip commas
pd.to_datetime(s, dayfirst=True, errors="coerce") # dd-mm-yyyy → date
df.fillna(0); df.dropna(subset=["k"])             # missing values
df.drop_duplicates(subset=["k"])                  # dedupe
df.duplicated(subset=["k"], keep=False)           # flag dupes

df.groupby("g").agg(total=("amt","sum"),
                    n=("id","count")).reset_index()
pd.merge(a, b, on="k", how="left", indicator=True)
np.where(cond, x, y)                              # if-else column
df.sort_values("amt", ascending=False)
df.to_excel("out.xlsx", index=False)              # export
```

Merge `how`	Keeps
`inner`	keys in **both**
`left`	**all left** + matching right
`right`	all right + matching left
`outer`	**everything**, NaN where unmatched

Python for Finance II: Automation

What it is & where it's used

Automation is using Python to do the boring, repeatable finance work that eats your day: pulling numbers out of Excel files, reshaping them, writing formatted output back to Excel, emailing the result, and running the whole thing on a schedule without you touching it. It is the difference between "I spend the first two hours of every month copy-pasting the MIS" and "the MIS lands in the CFO's inbox at 8:00 AM on the 1st while I'm still asleep."

The core libraries are **pandas** (read/transform/write tabular data), **openpyxl** (fine-grained Excel control — formatting, formulas, multiple sheets, cell styling), **smtplib**/`email` (send mail via SMTP), and a scheduler (Windows Task Scheduler or `cron` on Linux; libraries like `schedule` for in-process timing).

Roles that live on this: **FP&A analysts** (monthly MIS, variance packs), **accounts/AP-AR teams** (vendor ageing, reconciliations), **tax/GST executives** (GSTR-2B vs purchase register matching), **treasury** (daily bank position), **audit** (sampling, ledger scrutiny), and anyone who owns a recurring "report" that today is a manual Excel ritual.

The gap: why companies want this (and college didn't teach it)

An MBA teaches you *what* a variance analysis or a debtors ageing means. It never shows you that in a real company that report is regenerated **every single month from a messy dump** — the ERP exports a CSV with merged headers, three date formats, and vendor names spelled four ways. College assignments give you one clean dataset once. Industry gives you the same dirty dataset forever, on a deadline.

The specific gap: graduates can *build* a report once by hand in Excel but cannot *productionise* it. They don't know that a 90-minute manual task can become a 4-second script. Employers pay a premium for the person who says "I automated the AP ageing; it now runs itself" — that person just gave the team back 20 hours a month. This is the single most visible, promotable skill for a junior finance hire because the time saved is directly measurable.

What "proficient" looks like

A job-ready person can, unaided:

- Read one or many Excel/CSV files into pandas, including specifying the header row, sheet name, and dtypes.
- Clean real dirt: strip whitespace, fix dates, dedupe, handle blanks, map inconsistent names.
- Do the transform (group-by, pivot, merge/lookup across sheets) in pandas.
- Write a **formatted** multi-sheet Excel file — bold headers, number formats (`#,##0.00`), frozen panes, coloured totals, even live Excel formulas — not a raw dump.
- Send that file as an email attachment via SMTP with a templated body.
- Schedule the script to run daily/monthly and log whether it succeeded or failed.
- Make it robust: if the input file is missing, it emails an alert instead of crashing silently.

Hands-on: how to actually do it

Read Excel/CSV into pandas

```
import pandas as pd

# Read a specific sheet, telling pandas the header is on row 2 (0-indexed)
df = pd.read_excel("Purchase_Register_Jun26.xlsx",
                   sheet_name="Data", header=1,
                   dtype={"GSTIN": str, "Invoice_No": str})

# CSV with Indian date format
df = pd.read_csv("bank_stmt.csv", parse_dates=["Txn_Date"], dayfirst=True)
```

Clean the dirt (this is 60% of real work)

```
df.columns = df.columns.str.strip()                       # kill trailing spaces in headers
df["Vendor"] = df["Vendor"].str.strip().str.upper()       # normalise names
df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce")  # text → number, bad → NaN
df = df.dropna(subset=["Invoice_No"]).drop_duplicates()
df["Txn_Date"] = pd.to_datetime(df["Txn_Date"], dayfirst=True, errors="coerce")
```

The transform — group, pivot, lookup

```
# Debtors ageing buckets
today = pd.Timestamp("2026-06-30")
df["Days"] = (today - df["Invoice_Date"]).dt.days
bins   = [-1, 30, 60, 90, 10**6]
labels = ["0-30", "31-60", "61-90", "90+"]
df["Bucket"] = pd.cut(df["Days"], bins=bins, labels=labels)

ageing = df.pivot_table(index="Customer", columns="Bucket",
                        values="Outstanding", aggfunc="sum",
                        fill_value=0, margins=True, margins_name="Total")

# VLOOKUP-equivalent: merge master data onto transactions
out = txns.merge(master[["Vendor", "GSTIN", "PAN"]], on="Vendor", how="left")
```

Write a *formatted* Excel workbook with openpyxl

```
from openpyxl.styles import Font, PatternFill, Alignment, numbers

with pd.ExcelWriter("MIS_Jun26.xlsx", engine="openpyxl") as xl:
    ageing.to_excel(xl, sheet_name="Ageing")
    ws = xl.sheets["Ageing"]

    # Bold header row + fill colour
    hdr = Font(bold=True, color="FFFFFF")
    fill = PatternFill("solid", fgColor="1F4E78")
    for cell in ws[1]:
        cell.font, cell.fill = hdr, fill
        cell.alignment = Alignment(horizontal="center")

    # Indian number format on all data columns
    for row in ws.iter_rows(min_row=2, min_col=2):
        for c in row:
            c.number_format = '#,##0.00'

    ws.freeze_panes = "B2"                # freeze header + first column
    ws.column_dimensions["A"].width = 30
```

You can even write **live Excel formulas** so the recipient sees a working sheet:

```
ws["F2"] = "=SUM(B2:E2)"     # openpyxl writes the formula string; Excel evaluates it
```

Send it by email (SMTP)

```
import smtplib, ssl
from email.message import EmailMessage

msg = EmailMessage()
msg["Subject"] = "Debtors Ageing – June 2026"
msg["From"]    = "mis-bot@company.com"
msg["To"]      = "cfo@company.com, controller@company.com"
msg.set_content("Hi team,\n\nPlease find the June debtors ageing attached.\n\nAuto-generated b

with open("MIS_Jun26.xlsx", "rb") as f:
    msg.add_attachment(f.read(), maintype="application",
        subtype="vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        filename="MIS_Jun26.xlsx")

with smtplib.SMTP_SSL("smtp.gmail.com", 465, context=ssl.create_default_context()) as s:
    s.login("mis-bot@company.com", APP_PASSWORD)   # Gmail: use an App Password, never your lo
    s.send_message(msg)
```

> Security note: never hard-code passwords. Read from an environment variable: `APP_PASSWORD = os.environ["MAIL_PW"]`. For Gmail/Outlook you must generate an **App Password** (2FA required).

Schedule it

Windows Task Scheduler (most Indian finance desktops are Windows):

1. Wrap your script in a `.bat`: `C:\Python\python.exe C:\scripts\mis.py >> C:\scripts\log.txt 2>&1`
2. Task Scheduler → Create Task → Triggers → *Monthly, day 1, 08:00* → Actions → *Start a program* → point to the `.bat`.
3. Tick "Run whether user is logged on or not."

Linux/server (`cron`) — run 8 AM on the 1st of every month:

```
0 8 1 * *  /usr/bin/python3 /home/fin/mis.py >> /home/fin/mis.log 2>&1
```

Worked example / mini-project: month-end vendor ageing bot

Goal: every month, read the AP ledger export, build a vendor ageing report bucketed by days overdue, format it, and email it to the finance head.

Input `AP_Ledger.xlsx` (realistic ₹ data):

Vendor	Invoice_No	Invoice_Date	Outstanding
ACME STEEL	INV-1001	2026-04-02	2,45,000
ACME STEEL	INV-1044	2026-05-28	1,10,000
BLUE LOGISTICS	INV-2210	2026-03-15	88,500
ZENITH PACK	INV-3300	2026-06-20	3,20,000

```
import pandas as pd, os, smtplib, ssl
from email.message import EmailMessage
from openpyxl.styles import Font, PatternFill

FILE = "AP_Ledger.xlsx"
if not os.path.exists(FILE):
    raise SystemExit("Input missing — alert should fire here")

df = pd.read_excel(FILE, dtype={"Invoice_No": str})
df["Invoice_Date"] = pd.to_datetime(df["Invoice_Date"])
df["Outstanding"]  = pd.to_numeric(df["Outstanding"], errors="coerce")

asof = pd.Timestamp("2026-06-30")
df["Days"]   = (asof - df["Invoice_Date"]).dt.days
df["Bucket"] = pd.cut(df["Days"], [-1,30,60,90,10**6],
                      labels=["0-30","31-60","61-90","90+"])

report = df.pivot_table(index="Vendor", columns="Bucket", values="Outstanding",
                        aggfunc="sum", fill_value=0, margins=True, margins_name="Total")

with pd.ExcelWriter("Vendor_Ageing_Jun26.xlsx", engine="openpyxl") as xl:
    report.to_excel(xl, sheet_name="Ageing")
    ws = xl.sheets["Ageing"]
    for c in ws[1]:
        c.font = Font(bold=True, color="FFFFFF")
        c.fill = PatternFill("solid", fgColor="1F4E78")
    for row in ws.iter_rows(min_row=2, min_col=2):
        for c in row: c.number_format = '#,##0'
    ws.freeze_panes = "B2"; ws.column_dimensions["A"].width = 28
```

Expected output (₹):

Vendor	0-30	31-60	61-90	90+	Total
ACME STEEL	0	1,10,000	0	2,45,000	3,55,000
BLUE LOGISTICS	0	0	0	88,500	88,500
ZENITH PACK	3,20,000	0	0	0	3,20,000
Total	3,20,000	1,10,000	0	3,33,500	7,63,500

Then attach and email exactly as in the SMTP block above. Point Task Scheduler at it for the 1st of each month — you never touch the ageing report again.

How it's tested

Interview questions:

- "The ERP export has the header on row 3 and dates as `DD-MM-YYYY`. How do you read it correctly?" (Answer: `read_excel(header=2)` + `pd.to_datetime( ... , dayfirst=True)`.)
- "How is `pd.merge(how='left')` different from an Excel VLOOKUP? What happens to non-matches?" (Left keeps all left rows; misses become `NaN`.)
- "How would you email a formatted Excel file every Monday without a person running it?" (SMTP + Task Scheduler/cron.)
- "How do you keep the SMTP password out of the code?" (Env var / secrets manager, App Password.)

Practical test: A timed take-home or on-screen task — "Here's a 5,000-row messy sales dump. Produce a clean, formatted Excel MIS with monthly totals by region, and a Python script we can re-run next month." They score you on: does it re-run without editing, is the output *formatted* (not raw), did you handle blank/duplicate rows, and did you make it fail gracefully.

Common mistakes & how pros avoid them

Mistake	Fix
Hard-coding file paths/dates so it breaks next month	Use `datetime.now()` and relative/config-driven paths
Delivering a raw pandas dump with no formatting	Always style headers, set `#,##0` formats, freeze panes
Password in the script, pushed to Git	Env vars; `.gitignore` secrets; App Passwords
Script crashes silently at 8 AM, nobody notices	Wrap in `try/except`, log to file, email an alert on failure
`to_excel` on a filename that's open in Excel → PermissionError	Close the file / write to a timestamped name
Ignoring dtype — GSTIN `27ABC...` read as float, invoice `007` loses leading zeros	Pass `dtype=str` for IDs/codes
Chained slow loops over rows	Vectorise with pandas group-by/ `cut`, not `for` loops

Learn-it roadmap & resources

Time to job-ready: 4–6 weeks if you already know basic Python (Chapter 04).

- **Week 1–2:** pandas read/clean/transform. Rebuild an old Excel report of yours in pandas.
- **Week 3:** openpyxl formatting + writing multi-sheet workbooks with formulas.
- **Week 4:** smtplib email + scheduling; convert one real recurring task into a scheduled bot.
- **Week 5–6:** robustness — logging, try/except, config files; build the ageing/MIS mini-project end-to-end.

Resources:

- *Automate the Boring Stuff with Python* (Al Sweigart) — free online; the Excel + email chapters are exactly this.
- **pandas** docs "10 minutes to pandas"; **openpyxl** official tutorial (both free).
- YouTube: "Python Excel automation" walkthroughs.
- Certification: none is required, but **Microsoft PL-300** (Power BI) and Google's Python courses on Coursera signal seriousness. The real credential is a GitHub repo showing a working MIS bot.

Quick-reference

Task	Snippet
Read Excel sheet	`pd.read_excel(f, sheet_name="Data", header=1)`
Read CSV, IN dates	`pd.read_csv(f, parse_dates=["d"], dayfirst=True)`
Force text dtype	`dtype={"GSTIN": str}`
Clean headers	`df.columns = df.columns.str.strip()`
Text → number	`pd.to_numeric(s, errors="coerce")`
Ageing buckets	`pd.cut(days, bins, labels= ... )`
VLOOKUP	`a.merge(b, on="key", how="left")`
Pivot with totals	`pivot_table( ... , margins=True)`
Write Excel	`df.to_excel("out.xlsx", index=False)`
Bold header (openpyxl)	`cell.font = Font(bold=True)`
₹ number format	`cell.number_format = '#,##0.00'`
Freeze panes	`ws.freeze_panes = "B2"`
Send mail	`smtplib.SMTP_SSL("smtp.gmail.com", 465)`
Secret from env	`os.environ["MAIL_PW"]`
Schedule (cron)	`0 8 1 * * python mis.py`
Schedule (Windows)	Task Scheduler → Monthly → run `.bat`

Python for Finance III: Analysis

What it is & where it's used

This chapter is where Python stops being a fancy calculator and starts *answering business questions that have money attached*: **What will next quarter's revenue be? How risky is this portfolio? What actually drives our costs?**

Four techniques carry most of the load in a finance/FP&A/analytics seat:

Technique	The question it answers	Library
Regression	"How much does X move Y?" (drivers, sensitivity, beta)	`statsmodels`, `scikit-learn`
Time-series / forecasting	"What's the number for next month?"	`statsmodels` (ARIMA, ETS)
Monte Carlo simulation	"What's the *range* of outcomes, not one guess?"	`numpy`
Simple trend/seasonality decomposition	"Is this growth real or just December?"	`statsmodels`

Who pays for this: **FP&A analysts** (revenue/expense forecasts), **credit & risk teams** (default probability, VaR), **treasury** (cash-flow forecasting), **equity research** (beta, factor models), **corporate finance** (DCF with simulated scenarios). In India, this is the difference between a ₹6 LPA "Excel MIS" analyst and a ₹14–20 LPA "FP&A / analytics" analyst doing the same numbers with rigour.

The gap: why companies want this (and college didn't teach it)

Your MBA taught you *regression theory* (R^2, t-stat, p-value) on a clean dataset in a stats exam, and *forecasting* as "take a growth rate and extrapolate." Industry needs the opposite skill: messy monthly data, and a defensible number you can put in a board deck.

The specific gaps:

- **College gave you point estimates; the business wants ranges.** "Revenue will be ₹52 Cr" is a liability. "₹48–56 Cr, 80% confidence" is analysis. Monte Carlo closes this and is almost never taught.
- **You learned regression to *test a hypothesis*; finance uses it to *drive a model*.** Nobody in FP&A cares about your p-value on a slide — they care that "every ₹1 of ad spend returns ₹3.20 of revenue, and here's the confidence interval."
- **Excel's forecast tools are a black box.** `FORECAST.ETS` gives a number with no diagnostics. When the CFO asks "why is the forecast flat?", you need to *see* the trend and seasonal components — statsmodels shows them.
- **Seasonality is ignored.** Freshers extrapolate a December spike into a full-year run-rate. Decomposition prevents this embarrassing mistake.

What "proficient" looks like

A job-ready person can, unaided:

- Load a monthly revenue/cost series into a pandas DataFrame with a proper `DatetimeIndex`.
- Fit an **OLS regression**, read the coefficients *as business levers*, and quote the confidence interval — not just recite R^2.
- Decompose a series into **trend + seasonality + residual** and say whether growth is structural.
- Fit a **Holt-Winters (ETS)** or **ARIMA** model and produce a forecast *with a prediction interval*.
- Build a **Monte Carlo** simulation of an NPV, portfolio return, or budget line and report P10/P50/P90.
- Say out loud what could make the forecast wrong (regime change, one-off events).

The bar is *interpretation*, not memorising library syntax. You can Google `.fit()`; you can't Google "what this coefficient means for our pricing."

Hands-on: how to actually do it

Setup:

```
import pandas as pd, numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

1. Regression (drivers & sensitivity)

Say you want to know how **ad spend** and **price discount** drive **units sold**.

```
df = pd.DataFrame({
    "units":    [1200, 1500, 1400, 1800, 2100, 1950, 2300, 2500],
    "adspend":  [50,   65,   60,   80,   95,   88,   110,  120],   # ₹ '000
    "discount": [5,    8,    6,    10,   12,   9,    14,   15],     # %
})

model = smf.ols("units ~ adspend + discount", data=df).fit()
print(model.summary())
print(model.params)          # the business levers
print(model.conf_int())      # 95% confidence interval per driver
```

Read it like this: the `adspend` coefficient (say `14.2`) means **each extra ₹1,000 of ad spend sells ~14 more units**. That sentence is what goes in the deck. `model.pvalues` under 0.05 = the driver is statistically real; `model.rsquared` = share of variation explained.

Predict for a planned scenario:

```
scenario = pd.DataFrame({"adspend": [130], "discount": [16]})
model.get_prediction(scenario).summary_frame(alpha=0.20)  # 80% interval
```

2. Time-series decomposition (is the trend real?)

```
rev = pd.read_csv("monthly_revenue.csv", parse_dates=["month"], index_col="month")
rev = rev.asfreq("MS")   # month-start frequency — statsmodels needs this

from statsmodels.tsa.seasonal import seasonal_decompose
dec = seasonal_decompose(rev["revenue"], model="additive", period=12)
dec.trend; dec.seasonal; dec.resid    # plot with dec.plot()
```

If `dec.trend` slopes up while `dec.seasonal` shows a December bump, you know the growth is structural and December is just seasonal — don't annualise December.

3. Forecasting with Holt-Winters (ETS)

Best default for monthly business data with trend + seasonality:

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

fit = ExponentialSmoothing(
    rev["revenue"], trend="add", seasonal="add", seasonal_periods=12
).fit()

forecast = fit.forecast(6)        # next 6 months
print(forecast.round(0))
```

For a prediction interval, use ARIMA/SARIMAX:

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
sar = SARIMAX(rev["revenue"], order=(1,1,1),
              seasonal_order=(1,1,1,12)).fit(disp=False)
pred = sar.get_forecast(6)
pred.predicted_mean                    # the forecast
pred.conf_int(alpha=0.20)              # 80% band — this is what CFOs want
```

4. Monte Carlo simulation (the range, not the point)

Simulate NPV of a project where revenue and cost are uncertain:

```
np.random.seed(42)
N = 50_000
rev  = np.random.normal(500, 60, N)      # ₹ Lakh, mean 500, sd 60
cost = np.random.normal(320, 40, N)      # ₹ Lakh
disc = 0.12
years = 5

cashflow = rev - cost                     # annual, ₹ Lakh
npv = sum(cashflow / (1+disc)**t for t in range(1, years+1))

print(f"P10: ₹{np.percentile(npv,10):.0f} L")
print(f"P50: ₹{np.percentile(npv,50):.0f} L")
print(f"P90: ₹{np.percentile(npv,90):.0f} L")
print(f"P(NPV<0): {(npv<0).mean():.1%}")   # probability of loss
```

That last line — **"11% chance this project loses money"** — is worth more than any single-point NPV.

Worked example / mini-project: forecast + risk a ₹ retail chain's revenue

You run FP&A for a 20-store apparel chain. You have 36 months of revenue and need a **12-month forecast with a risk band** for the annual budget.

Step 1 — build the data (reproducible):

```
import pandas as pd, numpy as np
idx = pd.date_range("2023-01-01", periods=36, freq="MS")
np.random.seed(7)
trend = np.linspace(180, 260, 36)                       # ₹ Lakh, growing
season = 40*np.sin(np.arange(36)*2*np.pi/12) + \
         np.where(pd.Series(idx).dt.month==10, 60, 0)   # Diwali (Oct) spike
noise = np.random.normal(0, 10, 36)
rev = pd.Series((trend+season+noise).round(1), index=idx, name="revenue").to_frame()
rev = rev.asfreq("MS")
```

Step 2 — decompose to confirm the Diwali seasonality:

```
from statsmodels.tsa.seasonal import seasonal_decompose
seasonal_decompose(rev["revenue"], model="additive", period=12).seasonal.head(12)
# October seasonal factor is large & positive → real festive uplift
```

Step 3 — forecast next 12 months with an 80% band:

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
m = SARIMAX(rev["revenue"], order=(1,1,1), seasonal_order=(1,1,0,12)).fit(disp=False)
fc = m.get_forecast(12)
out = pd.concat([fc.predicted_mean.rename("base"),
                 fc.conf_int(alpha=0.20)], axis=1).round(0)
print(out)
annual_base = fc.predicted_mean.sum()
print(f"FY budget (base): ₹{annual_base:.0f} L")
```

Step 4 — Monte Carlo the annual total (because point forecasts lie):

```
resid_sd = m.resid.std()
sims = fc.predicted_mean.values[:, None] + \
       np.random.normal(0, resid_sd, (12, 20_000))
annual = sims.sum(axis=0)
print(f"Budget P10 ₹{np.percentile(annual,10):.0f} L | "
      f"P50 ₹{np.percentile(annual,50):.0f} L | "
      f"P90 ₹{np.percentile(annual,90):.0f} L")
```

The deliverable sentence: "*FY revenue budget of ₹3,180 L (base), with an 80% range of ₹2,980–3,390 L. October carries a ~₹60 L festive uplift; a stretch target above ₹3,390 L is < 10% likely.*" That is FP&A, not MIS.

How it's tested

Interview questions:

- "Difference between R^2 and adjusted R^2? Why prefer adjusted when adding variables?"
- "Your forecast says revenue drops next month but the business is growing — how do you debug it?" (Answer: check seasonality, check for a stationarity/differencing issue, look at residuals.)
- "What's a prediction interval and why is it more useful than a point forecast to a CFO?"
- "You have monthly sales for 2 years. Walk me through building a forecast." (They want: inspect → decompose → choose ETS/ARIMA → validate on a hold-out → forecast with interval.)
- "Explain Monte Carlo to a non-technical CEO in 30 seconds."

Practical/take-home tests:

- A CSV of 24–48 months of revenue: "*Forecast the next 6 months and justify your method.*" Graded on validation (did you hold out the last 6 months and check error?), not on the model name.
- A regression screen: "*Here are marketing spend and sales — quantify the ROI per rupee and state your confidence.*"
- A risk case: "*Given these cost/price distributions, what's the probability this launch is NPV-negative?*" — pure Monte Carlo.

Pro move in any test: **always back-test.** Fit on data up to month N-6, forecast the last 6 months you *do* have, and report MAPE. Showing `mean(abs((actual-pred)/actual))` proves the model, and freshers never do it.

Common mistakes & how pros avoid them

Mistake	Why it's wrong	The fix
No `DatetimeIndex` / no `.asfreq()`	statsmodels forecasts silently break or mis-align	Always `parse_dates`, set index, `.asfreq("MS")`
Reporting a point forecast only	Hides all risk; CFO gets blindsided	Always attach a prediction interval / P10–P90
Extrapolating a seasonal spike	Annualising Diwali/December = wild over-forecast	Decompose first; forecast on de-seasonalised logic
Confusing correlation with driver	"Ad spend correlates" ≠ "ad spend causes"	State assumptions; watch for omitted variables
Overfitting ARIMA orders	Great fit, garbage forecast	Back-test on hold-out; prefer simple orders
Monte Carlo with wrong distribution	Normal on bounded/skewed data lies	Use lognormal/triangular where appropriate; sanity-check tails
Not setting a random seed	Results change every run; not reproducible	`np.random.seed(...)` in any deliverable

Learn-it roadmap & resources

Realistic time-to-proficiency (assuming you can already do pandas from Chapters 04–05):

Phase	Time	Milestone
Regression with statsmodels	1 week	Read a `.summary()`, interpret coefficients as business levers
Decomposition + ETS/ARIMA	2 weeks	Forecast a real monthly series with a validated hold-out
Monte Carlo	3–4 days	Simulate an NPV/portfolio, report P10/P50/P90
Portfolio project	1 week	End-to-end forecast+risk memo on your own data

≈ 5–6 weeks part-time to interview-ready.

Resources (free-first):

- **statsmodels docs** — the time-series and OLS example galleries are excellent and free.
- **"Forecasting: Principles and Practice" (Hyndman)** — free online, the standard text. It's in R but the concepts (ETS, ARIMA, back-testing) map 1:1 to statsmodels.
- **Kaggle** — "Store Sales" / retail time-series datasets to practice on real messy data.
- **QuantEcon** (free) — for Monte Carlo and numpy-heavy finance.
- **Certification worth having:** none specific to this; a **Google Data Analytics** or **CFA Level I quant** signal helps in India, but a GitHub repo with one clean forecast+Monte-Carlo notebook beats any cert in interviews.

Quick-reference

```
# --- Regression ---
import statsmodels.formula.api as smf
m = smf.ols("y ~ x1 + x2", data=df).fit()
m.summary(); m.params; m.pvalues; m.conf_int(); m.rsquared_adj
m.get_prediction(new_df).summary_frame(alpha=0.20)   # 80% interval

# --- Prep a series ---
s = df.set_index("month").asfreq("MS")["revenue"]

# --- Decompose ---
from statsmodels.tsa.seasonal import seasonal_decompose
seasonal_decompose(s, model="additive", period=12).plot()

# --- Forecast: Holt-Winters ---
from statsmodels.tsa.holtwinters import ExponentialSmoothing
ExponentialSmoothing(s, trend="add", seasonal="add",
                     seasonal_periods=12).fit().forecast(6)

# --- Forecast: SARIMAX (with interval) ---
from statsmodels.tsa.statespace.sarimax import SARIMAX
f = SARIMAX(s, order=(1,1,1), seasonal_order=(1,1,1,12)).fit(disp=False)
f.get_forecast(6).predicted_mean
f.get_forecast(6).conf_int(alpha=0.20)

# --- Monte Carlo NPV ---
np.random.seed(42)
cf = np.random.normal(180, 30, 50_000)
npv = sum(cf/(1.12)**t for t in range(1,6))
np.percentile(npv,[10,50,90]); (npv<0).mean()

# --- Back-test (always do this) ---
train, test = s[:-6], s[-6:]
pred = ExponentialSmoothing(train, trend="add", seasonal="add",
        seasonal_periods=12).fit().forecast(6)
mape = (abs((test-pred)/test)).mean()   # < 0.10 is good
```

Concept	Rule of thumb
R^2 vs adj-R^2	Use adjusted when comparing models with different # of variables
p-value	< 0.05 → driver is statistically real
Prediction interval	Always report; `alpha=0.20` = 80% band
MAPE	< 10% good, 10–20% acceptable, > 20% rethink
Monte Carlo output	Report P10 / P50 / P90 + probability of loss
Seasonality	Decompose before forecasting monthly data
Reproducibility	`np.random.seed()` on every deliverable

Power BI I: Data Model & DAX

What it is & where it's used

Power BI is Microsoft's business-intelligence tool: you load data, build a **data model** (tables + relationships), write **DAX** (Data Analysis Expressions) formulas, and publish interactive dashboards. Think of it as "Excel PivotTables on steroids" — it handles 10 million rows without choking, refreshes on a schedule, and lets a CFO slice revenue by region/month/product with one click.

Two separate skills live inside Power BI, and this chapter covers the engine, not the paint: - **Power Query (M)** — the ETL layer: import, clean, reshape. - **The data model + DAX** — relationships, star schema, and the calculation language.

Where finance roles use it:

Role	What they build in Power BI
FP&A analyst	Monthly MIS, budget-vs-actuals, rolling forecasts
Accounts / controller	AR/AP ageing, cash-flow dashboards, GST reconciliation summaries
Audit / internal audit	Exception reports, journal-entry testing, sampling views
Business finance	Revenue by SKU/region, margin walk, cohort analysis
Tax	GSTR-2B vs purchase-register match, ITC tracking

In India, "Power BI" now appears in ~30-40% of FP&A and MIS job posts, often paired with Excel and SQL. It is the single highest-leverage BI skill you can add to a finance CV.

The gap: why companies want this (and college didn't teach it)

MBA Finance and CA teach you *what* the numbers mean — variance analysis, ratios, cost sheets. They do **not** teach you to model data. The specific gaps:

1. **You think in one big flat table.** College spreadsheets have everything in one sheet. Real reporting needs a **star schema** — separate fact and dimension tables joined by keys. Nobody explains why.
2. **You confuse a column with a measure.** In Excel every cell is a value. In Power BI a *measure* is a formula that recalculates for whatever the user clicked. Freshers write everything as calculated columns and blow up the file size.
3. **You've never seen "filter context."** DAX's entire personality is that `CALCULATE` manipulates filters. This concept has no Excel equivalent, so it's the #1 thing people fail.
4. **Time-intelligence.** "Show me YTD, same period last year, MoM %" — done in Excel with fragile SUMIFS and helper columns. Power BI does it in one measure *if you have a proper date table*.

Employers pay for this because a good model turns a 3-day monthly-MIS grind into a 5-minute refresh. That is a direct cost saving they can measure.

What "proficient" looks like

A job-ready person can, unaided:

- Load 3-4 tables, set correct **data types**, and build a **star schema** with one-to-many relationships.
- Explain when to use a **calculated column** vs a **measure** (and default to measures).
- Create a **dedicated Date table** and mark it as such.
- Write measures using `SUM`, `SUMX`, `CALCULATE`, `DIVIDE`, `FILTER`, and the time-intelligence functions (`TOTALYTD`, `SAMEPERIODLASTYEAR`, `DATEADD`).
- Understand and control **filter context** — knows why a measure returns blank or the grand total in the wrong cell.
- Build a budget-vs-actual with variance and variance %.

That's roughly the bar for an FP&A analyst screen. You do **not** need row-level security or DAX Studio optimization for a first job.

Hands-on: how to actually do it

1. Load data

`Home → Get Data → Excel/CSV/SQL Server`. Pick your files, click **Transform Data** (opens Power Query), verify each column's data type (the icon left of the header), then **Close & Apply**.

Golden rule: **clean in Power Query, calculate in DAX.** Don't do transformations with DAX columns.

2. Build the star schema

A star schema = one **fact** table (transactions, many rows) surrounded by **dimension** tables (master data, few rows).

```
        Dim_Date
            |
Dim_Product — Fact_Sales — Dim_Customer
            |
        Dim_Region
```

- **Fact_Sales**: Date, ProductKey, CustomerKey, RegionKey, Qty, Amount.
- **Dimensions**: one row per product/customer/region/date.

In **Model view**, drag `Fact_Sales[ProductKey]` onto `Dim_Product[ProductKey]`. Power BI creates a **one-to-many** relationship (one product → many sales rows). Filters flow **from the "one" side to the "many" side** — this single fact explains 90% of DAX behaviour.

3. Measures vs columns

	Calculated column	Measure
Computed	Row-by-row, at refresh	On the fly, per filter context
Stored	In the model (eats RAM)	Not stored
Use for	A category/flag you slice BY	A number you aggregate/show
Example	`Margin Band = IF([Margin]>0.3,"High","Low")`	`Total Sales = SUM(Fact_Sales[Amount])`

Default to measures. Only make a column when you need to *filter or group by* the result.

4. Core DAX

```
Total Sales = SUM ( Fact_Sales[Amount] )

Total Qty = SUM ( Fact_Sales[Qty] )

-- SUMX iterates row by row, then sums. Use when the math is per-row.
Revenue = SUMX ( Fact_Sales, Fact_Sales[Qty] * Fact_Sales[Price] )

-- DIVIDE handles divide-by-zero (returns blank, not error)
Avg Price = DIVIDE ( [Revenue], [Total Qty] )
```

CALCULATE — the most important function. It evaluates an expression under *modified* filters:

```
-- Sales only for the South region, ignoring any region the user clicked
South Sales = CALCULATE ( [Total Sales], Dim_Region[Region] = "South" )

-- Sales ignoring ALL filters (useful for % of total)
All Sales = CALCULATE ( [Total Sales], ALL ( Fact_Sales ) )

Pct of Total = DIVIDE ( [Total Sales], [All Sales] )
```

Time-intelligence (needs a marked Date table):

```
-- Create a proper date table first
Dim_Date =
CALENDAR ( DATE ( 2023, 4, 1 ), DATE ( 2026, 3, 31 ) )
```

Add columns to it: `Year = YEAR([Date])`, `MonthNo = MONTH([Date])`, `MonthName = FORMAT([Date],"MMM")`, `FY = "FY" & YEAR([Date]) + IF(MONTH([Date])≥4,1,0)` (Indian April-March fiscal year). Then `Table tools → Mark as date table`.

```
Sales YTD = TOTALYTD ( [Total Sales], Dim_Date[Date], "31/03" )   -- Indian FY end

Sales LY = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( Dim_Date[Date] ) )

Sales PM = CALCULATE ( [Total Sales], DATEADD ( Dim_Date[Date], -1, MONTH ) )

MoM % = DIVIDE ( [Total Sales] - [Sales PM], [Sales PM] )

YoY % = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
```

Worked example / mini-project: Budget vs Actual MIS

Data (reproduce in two CSVs):

`Fact_Actuals` (sales transactions):

Date	Region	Product	Amount
2025-04-05	South	Widget A	120000
2025-04-18	North	Widget B	85000
2025-05-02	South	Widget A	140000
2025-05-20	West	Widget C	60000

`Fact_Budget` :

Month	Region	BudgetAmount
2025-04	South	130000
2025-04	North	90000
2025-05	South	150000
2025-05	West	55000

Steps: 1. Load both. Build `Dim_Date` and `Dim_Region` ; relate both facts to them. 2. Write measures:

```
Actual = SUM ( Fact_Actuals[Amount] )
Budget = SUM ( Fact_Budget[BudgetAmount] )
Variance = [Actual] - [Budget]
Variance % = DIVIDE ( [Variance], [Budget] )
Actual YTD = TOTALYTD ( [Actual], Dim_Date[Date], "31/03" )
```

3. Drop a matrix: Rows = `Region` , Values = `Actual` , `Budget` , `Variance` , `Variance %` . Add a `MonthName` slicer.

Expected April result: South Actual ₹1,20,000 vs Budget ₹1,30,000 → Variance −₹10,000 (−7.7%). North −₹5,000 (−5.6%). This is a live, filterable MIS you built in under 30 minutes — exactly the deliverable an FP&A

team asks a fresher to produce.

Conditional formatting: on `Variance %`, `Format → Cell elements → Background color → rules`: red below 0, green above. Now management sees misses at a glance.

How it's tested

Interview questions: - "Difference between a calculated column and a measure? When each?" - "What is a star schema and why not just one flat table?" - "Explain filter context. What does `CALCULATE` do?" - "Why do you need a separate date table instead of the date column in your fact?" - "`SUM` vs `SUMX` — give an example where only `SUMX` works." (Answer: `Qty * Price` per row, since prices differ by row.) - "Your measure shows blank / shows the grand total in every row — why?"

Practical test (very common): They hand you 2-3 raw CSVs/Excel files and a brief — "*Build a dashboard showing sales by region and month, with YoY growth and a budget-vs-actual variance*" — timed 45-90 minutes. Graders check: correct relationships (no many-to-many hacks), measures not columns, a proper date table, working time-intelligence, and clean visuals. Some firms give a broken .pbix and ask you to *fix the model*.

Common mistakes & how pros avoid them

Mistake	Fix
Everything in one flat table	Split into fact + dimensions (star schema)
Calculated columns for numbers you aggregate	Use measures — they respond to filters and don't bloat the file
Using the fact table's date column for time-intelligence	Always a **dedicated, marked Date table** with continuous dates
Gaps in the date table	Use `CALENDAR` / `CALENDARAUTO` so every day exists
`/` operator causing errors	Use `DIVIDE()` — safe divide-by-zero
Bi-directional / many-to-many relationships everywhere	Keep single-direction one-to-many; filter flows one → many
Hardcoding "last year" with a slicer	Use `SAMEPERIODLASTYEAR` / `DATEADD`
Wrong FY (Jan-Dec) for Indian data	Set year-end to `"31/03"` and build FY logic (Apr-Mar)
Blank measure results	Usually broken relationship or wrong filter direction — check Model view

Learn-it roadmap & resources

Time to proficiency: 4-6 weeks part-time to clear a fresher FP&A screen; 3 months to be genuinely fast.

Week	Focus
1	Power BI Desktop install (free), Get Data, Power Query cleaning
2	Model view, relationships, star schema
3	Measures vs columns, `SUM/SUMX/CALCULATE/DIVIDE`
4	Date table + time-intelligence (YTD, YoY, MoM)
5-6	Build 2 end-to-end dashboards; learn `FILTER`, `ALL`, `VAR`

Resources: - **Microsoft Learn — "Power BI Data Analyst" path** (free, official). - **SQLBI (Marco Russo/Alberto Ferrari)** — free YouTube + *dax.guide* function reference; the gold standard for DAX. - **Enterprise DNA** and **Kevin Stratvert** on YouTube (free, beginner-friendly). - **Certification: PL-300 (Microsoft Power BI Data Analyst)** — ~₹4,800 exam fee in India, well-recognised, worth it for finance CVs.

Power BI Desktop is **free**. Publishing to the cloud service needs Pro (~$10/month); for learning and interviews, Desktop alone is enough.

Quick-reference

```
-- Aggregation
SUM ( Table[Col] )
SUMX ( Table, Table[A] * Table[B] )      -- per-row then sum
DIVIDE ( num, den [, 0] )                -- safe divide

-- Filter manipulation
CALCULATE ( [Measure], filter1, filter2 )
CALCULATE ( [Measure], ALL ( Table ) )    -- remove filters
CALCULATE ( [Measure], FILTER ( Table, Table[x] > 100 ) )

-- Time-intelligence (needs marked Date table)
TOTALYTD ( [Measure], Dim_Date[Date], "31/03" )   -- Indian FY
CALCULATE ( [Measure], SAMEPERIODLASTYEAR ( Dim_Date[Date] ) )
CALCULATE ( [Measure], DATEADD ( Dim_Date[Date], -1, MONTH ) )
```

Concept	One-liner
Star schema	1 fact (transactions) + many dimensions (masters)
Relationship	One-to-many; filter flows one → many
Measure	Formula, recalculates per filter; use for numbers you show
Calculated column	Row-by-row, stored; use only to slice/group BY
Filter context	The set of filters active when a measure evaluates
CALCULATE	Changes the filter context, then evaluates
Date table	Dedicated, continuous, "Mark as date table" — required for time-intel
Indian FY	April 1 – March 31; set YTD year-end to `"31/03"`
``	

Muscle memory: clean in Power Query → model as a star → default to measures → one Date table → `CALCULATE` for everything conditional.

Power BI II: Finance Dashboards

What it is & where it's used

Chapter 07 covered the plumbing — Power Query, the star schema, the DAX basics. This chapter is where you turn that model into the three deliverables a finance team actually asks for: a **P&L (Profit & Loss) dashboard**, a **budget-vs-actual (BvA) variance report**, and a **KPI scorecard** — plus the two features that separate a "chart-maker" from a "BI person": **drill-through** and **row-level security (RLS)**.

Who builds these:

Role	What they ship in Power BI
FP&A analyst	Monthly BvA pack, rolling forecast, KPI board for the CFO
Management accountant	Cost-centre P&L, margin analysis, drill-through to cost lines
Financial analyst / MIS	Revenue/AR/AP dashboards, DSO/DPO trends
Controller / Finance Manager	Board deck built off one governed dataset, RLS by entity/region
Business finance partner	Region- or product-level P&L where each manager sees only their slice

In Indian companies (and MNC GCCs in Bengaluru/Hyderabad/Pune), the "monthly MIS pack" is very often a Power BI report refreshed off the ERP (SAP, Oracle, Tally exports, or a data warehouse). If you can own that pack, you are the finance-tech hire they cannot easily replace.

The gap: why companies want this (and college didn't teach it)

MBA and CA syllabi teach you to *read* a P&L and *compute* a variance. They stop there. Industry needs someone who can:

- Turn a raw ledger dump (10 lakh rows) into a **refreshable, filterable P&L** — not a static Excel that breaks every month-end.
- Build **variance % and favourable/adverse flags** that update automatically when a controller drops in new actuals.
- Let a Regional Head click a total and **drill through to the transactions behind it** without pinging the analyst.
- Show the **Sales VP only the South zone** and the **Finance Head everything** — from *one* report, using RLS, instead of maintaining 6 separate files.

College teaches the *accounting*; the employer pays for the *delivery mechanism*. Nobody in a BCom/MBA classroom builds a `DIVIDE()` variance measure or writes a DAX RLS filter. That is exactly the arbitrage.

What "proficient" looks like

A job-ready person can, unaided:

1. Build a **date-intelligence P&L** with subtotals (Revenue → Gross Profit → EBITDA → PBT) using measures, not hard-coded columns.

2. Write **variance measures** (Actual − Budget, Var %, favourable/adverse) that respect the sign convention (over-spend on cost is *adverse*, over-shoot on revenue is *favourable*).
3. Configure **drill-through** so right-click → "See transactions" jumps to a filtered detail page.
4. Set up **RLS roles**, write the DAX filter, and **test via "View as role."**
5. Use **conditional formatting** (red/green) and **KPI cards** that a CFO can read in 10 seconds.
6. Schedule a **refresh** and know the difference between Import and DirectQuery.

Hands-on: how to actually do it

Data model assumed

Star schema from Chapter 07: - `Fact_GL` (Date, AccountKey, CostCentreKey, Amount, Scenario) — Scenario = "Actual" or "Budget" - `Dim_Account` (AccountKey, Account, PLGroup, PLGroupSort, SignConvention) - `Dim_Date`, `Dim_CostCentre` (CostCentreKey, CostCentre, Region, Manager, ManagerEmail)

1. Core P&L measures (DAX)

```
Actual =
CALCULATE ( SUM ( Fact_GL[Amount] ), Fact_GL[Scenario] = "Actual" )

Budget =
CALCULATE ( SUM ( Fact_GL[Amount] ), Fact_GL[Scenario] = "Budget" )

-- Revenue and cost stored with natural signs (revenue +, expense −)
Revenue =
CALCULATE ( [Actual], Dim_Account[PLGroup] = "Revenue" )

Gross Profit =
CALCULATE ( [Actual],
    Dim_Account[PLGroup] IN { "Revenue", "COGS" } )

EBITDA =
CALCULATE ( [Actual],
    NOT Dim_Account[PLGroup] IN { "Depreciation", "Interest", "Tax" } )
```

Build the P&L statement itself with a **Matrix visual**: `Dim_Account[PLGroup]` on rows (sorted by `PLGroupSort`), `[Actual]` and `[Budget]` on values. This gives you a real statement layout instead of a bar chart.

2. Budget-vs-Actual variance

```
Variance = [Actual] - [Budget]

Variance % =
DIVIDE ( [Actual] - [Budget], [Budget] )   -- DIVIDE avoids /0 error

-- Sign-aware flag: cost overrun is adverse, revenue beat is favourable
Var Status =
VAR IsRevenue =
    SELECTEDVALUE ( Dim_Account[SignConvention] ) = "Income"
VAR Fav = IF ( IsRevenue, [Variance] ≥ 0, [Variance] ≤ 0 )
RETURN IF ( Fav, "Favourable", "Adverse" )
```

Conditional formatting on the Variance column: Format → Cell elements → Background color → *Rules* → if `Var Status` = "Adverse" then red, else green. Now the CFO sees the problem lines instantly.

3. KPI measures & cards

```
Gross Margin % = DIVIDE ( [Gross Profit], [Revenue] )

EBITDA Margin % = DIVIDE ( [EBITDA], [Revenue] )

-- Month-over-month growth using the date dimension
Revenue MoM % =
VAR Prev =
    CALCULATE ( [Revenue], DATEADD ( Dim_Date[Date], -1, MONTH ) )
RETURN DIVIDE ( [Revenue] - Prev, Prev )

Budget Achievement % = DIVIDE ( [Actual], [Budget] )
```

Drop each into a **Card** or **KPI visual**. For the KPI visual: Value = `[Revenue]`, Target = `[Budget]`, Trend axis = `Dim_Date[Month]`.

4. Drill-through

1. Add a new page, rename it `Transaction Detail`.
2. In the **Visualizations** → **Drill through** well, drag the field you want to drill *by*, e.g. `Dim_Account[Account]` (and/or `Dim_CostCentre[CostCentre]`).
3. Power BI auto-adds a "back" button. Put a Table visual on the page: Date, Account, Cost Centre, Amount, Voucher No.
4. On the summary P&L page, **right-click any total** → **Drill through** → **Transaction Detail.** The detail page opens filtered to that account.
5. Toggle **"Keep all filters" = On** so region/month context carries over.

5. Row-level security (RLS)

```
Modeling tab → Manage roles → Create
```

Role: **Region_Manager**

```
-- Filter Dim_CostCentre so each user sees only their region
[Region] =
    LOOKUPVALUE (
        UserMap[Region],
        UserMap[Email], USERPRINCIPALNAME ()
    )
```

Or the simpler "manager sees own cost centres" pattern:

```
-- Applied on Dim_CostCentre
Dim_CostCentre[ManagerEmail] = USERPRINCIPALNAME ()
```

Test before publishing: **Modeling → View as → tick Region_Manager**, optionally enter a test email under "Other user." The whole report re-filters. After **Publish**, in the Power BI Service go to the dataset → **Security** → add the Azure AD users/groups to the role. Without that last step, RLS does nothing in the Service.

> One `USERPRINCIPALNAME()` returns the logged-in user's email — that is what makes one report serve 50 managers.

Worked example / mini-project

Scenario: "Bharat Consumer Products Pvt Ltd" — FY 2025-26, three regions. You are handed a GL export. Reproduce this in ₹ lakhs.

`Fact_GL` (sample, Actual scenario, April):

Date	Account	PLGroup	CostCentre	Region	Scenario	Amount (₹ L)
2025-04-30	Product Sales	Revenue	CC-North	North	Actual	420
2025-04-30	Product Sales	Revenue	CC-South	South	Actual	510
2025-04-30	Raw Material	COGS	CC-North	North	Actual	−250
2025-04-30	Raw Material	COGS	CC-South	South	Actual	−290
2025-04-30	Salaries	Employee	CC-Head	HO	Actual	−95
2025-04-30	Freight	Distribution	CC-North	North	Actual	−38

Budget rows exist with Scenario = "Budget" (e.g. North Sales budget 400, South 480).

Build the P&L matrix (South region slicer applied):

P&L line	Actual	Budget	Variance	Var %	Status
Revenue	510	480	+30	+6.3%	Favourable
COGS	−290	−270	−20	+7.4%	Adverse
Gross Profit	**220**	**210**	**+10**	**+4.8%**	Favourable
Distribution	−22	−20	−2	+10%	Adverse
EBITDA (region)	**198**	**190**	**+8**	**+4.2%**	Favourable

KPI cards: Gross Margin % = 220/510 = **43.1%**; Budget Achievement (Rev) = 510/480 = **106.3%**.

Drill-through demo: the Regional Head clicks the COGS −290, drills through, and sees the Raw Material vouchers making up that number — spots that one supplier invoice caused the ₹20 L overrun.

RLS demo: publish with role `Region_Manager` on `ManagerEmail = USERPRINCIPALNAME()`. The South manager logs in → the entire report shows only 510 revenue, never North's 420. The Finance Head is in no role → sees the consolidated ₹930 revenue.

Reproduce it: paste the two tables into Excel, load via Get Data → Excel, mark `Dim_Date`, write the six measures above, and you have a working pack in ~30 minutes.

How it's tested

Interview questions: - "How do you build a P&L in Power BI without hard-coding rows for each line item?" (Answer: measures + Account dimension with a sort/group column.) - "Actuals came in over budget on marketing spend — how does your report show that as *adverse* while a revenue beat shows *favourable*?" (Sign convention logic.) - "Difference between a slicer filter and RLS?" (Slicer = user choice, removable; RLS = enforced at dataset, user cannot bypass.) - "Import vs DirectQuery — which for a monthly MIS pack?" (Import; faster, scheduled refresh.) - "What does `USERPRINCIPALNAME()` return and why does it matter for RLS?"

Practical/assessment test: Very common — a 45–60 min take-home or on-site. Typical brief: "*Here's a GL extract and a budget file. Build a P&L with Actual/Budget/Variance, add a Gross Margin KPI card, enable drill-through to transactions, and create an RLS role so a region manager sees only their region. Publish and share a test link.*" They grade: correct variance signs, no `/0` errors, working drill-through, and — the pass/fail line — whether RLS actually restricts data when they "View as role."

Common mistakes & how pros avoid them

Mistake	Fix
Hard-coding P&L rows as separate measures per line	Use one Account dimension + PLGroup + sort column; drive rows dynamically
`Variance % = Var / Budget` throwing errors on zero budget	Always use `DIVIDE()`
Variance sign wrong (cost overrun shown green)	Store a `SignConvention` and build a sign-aware `Var Status`
Building 6 separate files, one per region	One dataset + RLS
Forgetting to add users to the role in the Service	RLS in Desktop alone does nothing; assign members under dataset → Security
Totals in matrix look wrong ("the total isn't the sum of rows")	That's measure context — verify with a test filter, don't "fix" by summing columns
Drill-through page not filtered	Ensure the drill-by field is in the Drill through well and "Keep all filters" is On
Refresh fails after month-end	Use a parameterised folder/date path in Power Query, not a fixed filename

Learn-it roadmap & resources

Time to proficient: ~3–4 weeks part-time if you already did Chapter 07's modelling.

Week	Focus
1	P&L matrix + core measures; conditional formatting
2	Budget-vs-Actual, DIVIDE, sign-aware variance, KPI cards
3	Drill-through, bookmarks, tooltips, a clean CFO-ready layout
4	RLS (static + dynamic), publish to Service, scheduled refresh

Resources: - Microsoft Learn — "Power BI data analyst" learning path (free). - SQLBI (Marco Russo / Alberto Ferrari) — free articles/YouTube on variance, RLS, time intelligence. The gold standard for DAX. - Book: *The Definitive Guide to DAX* (Russo & Ferrari) for depth. - **Certification:** Microsoft **PL-300: Power BI Data Analyst Associate** — the one recruiters recognise; covers modelling, DAX, RLS, and publishing. ~₹4,800 exam fee in India, strongly worth listing on your CV.

Quick-reference

```
Actual   = CALCULATE(SUM(Fact_GL[Amount]), Fact_GL[Scenario]="Actual")
Budget   = CALCULATE(SUM(Fact_GL[Amount]), Fact_GL[Scenario]="Budget")
Variance = [Actual] - [Budget]
Var %    = DIVIDE([Actual]-[Budget], [Budget])
GM %     = DIVIDE([Gross Profit], [Revenue])
MoM %    = DIVIDE([Revenue]-CALCULATE([Revenue],DATEADD(Dim_Date[Date],-1,MONTH)),
                  CALCULATE([Revenue],DATEADD(Dim_Date[Date],-1,MONTH)))
RLS      = Dim_CostCentre[ManagerEmail] = USERPRINCIPALNAME()
```

Task	Click-path
P&L layout	Matrix visual → PLGroup on Rows, Actual/Budget on Values
Red/green variance	Format → Cell elements → Background color → Rules
Drill-through	Detail page → drag field to *Drill through* well → right-click total on summary
Create RLS role	Modeling → Manage roles → new role → DAX filter
Test RLS	Modeling → View as → tick role
Activate RLS live	Service → dataset → Security → add users to role
Refresh	Service → dataset → Schedule refresh (Import mode)

Sign rule: Income favourable when Actual ≥ Budget; Expense favourable when Actual ≤ Budget. Always `DIVIDE()`, never `/`. One dataset + RLS beats many files, every time.

Tableau & Choosing Your BI Tool

What it is & where it's used

Tableau is a drag-and-drop data-visualization and dashboarding tool. You connect it to a data source (Excel, CSV, SQL Server, Snowflake, Google Sheets), drag fields onto "shelves," and it draws charts. String a few charts together and you have an interactive dashboard that a CFO can filter by region, month, or product without touching a formula.

In finance/accounts roles, BI tools live wherever numbers need to be *seen* repeatedly by non-analysts:

Role	What they build in Tableau/Power BI
FP&A analyst	Budget vs actual dashboard, refreshed monthly
Revenue/AR analyst	Ageing dashboard, DSO trend, collection heatmap
Cost/plant controller	Cost-per-unit, variance waterfall by cost centre
Treasury	Cash-position dashboard across bank accounts
Tax/compliance	GST liability vs ITC trend, filing-status tracker
MIS executive	The monthly management deck, auto-refreshed

The pattern is identical everywhere: raw data → clean model → chart → published dashboard that others self-serve. Tableau is the market-leading tool for the "chart + dashboard" half. Power BI is its main rival. Excel is the third leg — and still the one you'll actually use most days.

The gap: why companies want this (and college didn't teach it)

An MBA teaches you to *analyse* a company. It does not teach you to build a thing that 40 people open every Monday. The gap is **repeatable, self-service reporting**:

- College output = a one-off answer in a slide. Industry output = a *living dashboard* that refreshes itself and lets the viewer drill down without emailing you.
- Nobody in a B-school case study asks "how will this refresh next month?" or "can the regional head filter to just South zone?" Every real MIS job asks exactly that.
- Excel charts break when data grows or columns move. BI tools separate the *data model* from the *view*, so the report survives new months of data. That architectural idea — model once, visualise many — is the actual skill, and it's never taught.

Companies pay for BI because manual monthly reporting is a tax: a junior spends three days rebuilding the same deck. A good dashboard turns three days into a five-minute refresh. That saving is the job.

What "proficient" looks like

The bar an employer tests for — what a job-ready person does unaided:

- **Connect and shape:** pull data from Excel/SQL, fix data types, create a clean join or relationship, without needing pre-cleaned data.

- **Calculated fields:** write a profit-margin, YoY-growth, or running-total calc from scratch.
- **The right chart:** knows a trend = line, a part-to-whole = bar (not pie), a variance = waterfall or bar with reference line. Doesn't decorate.
- **Interactivity:** adds filters, parameters, and drill-downs so the viewer answers their own follow-up.
- **Publish & control access:** pushes to Tableau Server/Cloud or Power BI Service, sets a refresh schedule, and shares to the right people (not "everyone in company").
- **Tool judgment:** can say *why* this report is Tableau and that one is just an Excel PivotTable — and not over-engineer.

Hands-on: how to actually do it

1. Connecting data (Tableau Public / Desktop) `Connect → Microsoft Excel → select file → drag sheet to canvas`. Tableau splits fields into **Dimensions** (text/date — the "by what") and **Measures** (numbers — the "how much"). Sales *by* Region *over* Month: drag `Order Date` to Columns, `Sales` to Rows, `Region` to Colour.

2. A calculated field (`Analysis → Create Calculated Field`):

```
// Profit Margin %
SUM([Profit]) / SUM([Sales])
```

```
// YoY Sales Growth (table calc alternative)
(SUM([Sales]) - LOOKUP(SUM([Sales]), -1)) / ABS(LOOKUP(SUM([Sales]), -1))
```

```
// Bucketing AR ageing
IF [Days Overdue] ≤ 30 THEN "0-30"
ELSEIF [Days Overdue] ≤ 60 THEN "31-60"
ELSEIF [Days Overdue] ≤ 90 THEN "61-90"
ELSE "90+" END
```

3. LOD expression (Tableau's signature feature — compute at a different grain than the view):

```
// Sales per customer, regardless of what's on the view
{ FIXED [Customer Name] : SUM([Sales]) }
```

4. The Power BI / DAX equivalent (same logic, different language). In Power BI you write measures:

```
Total Sales = SUM(Sales[Amount])

Profit Margin % =
DIVIDE(SUM(Sales[Profit]), SUM(Sales[Amount]))

YoY Growth % =
VAR CurYr = [Total Sales]
VAR PrevYr = CALCULATE([Total Sales], DATEADD('Date'[Date], -1, YEAR))
RETURN DIVIDE(CurYr - PrevYr, PrevYr)
```

5. The Excel equivalent (for when a dashboard is overkill). A PivotTable + slicer *is* a mini-BI dashboard. To pull a value into a management sheet:

```
=XLOOKUP(A2, Data!$A:$A, Data!$D:$D, "Not found")

=SUMIFS(Sales[Amount], Sales[Region], "South", Sales[Month], "Apr")
```

6. Publishing (Tableau): `Server → Publish Workbook → sign in to Tableau Cloud → choose Project (folder) → set Permissions → set Data Source refresh schedule (e.g. daily 7am)`. Viewers open a URL; no Tableau install needed.

7. Publishing (Power BI): `Home → Publish → pick Workspace`. Then in the browser: `Workspace → Dataset → Schedule refresh → set gateway + time`. Share via `App` or add users to the workspace.

Worked example / mini-project

Build: an Accounts Receivable ageing dashboard for an Indian mid-size firm.

Sample data (`invoices.csv`), reproduce with ~200 rows like:

Invoice	Customer	Region	Invoice Date	Due Date	Amount (₹)	Paid?
INV-1001	Reliance Retail	West	2026-04-05	2026-05-05	4,50,000	No
INV-1002	Infosys	South	2026-05-12	2026-06-11	2,20,000	No
INV-1003	Tata Steel	East	2026-03-01	2026-03-31	8,75,000	No

Steps:

1. Connect the CSV in Tableau. Create `Days Overdue = DATEDIFF('day', [Due Date], TODAY())`.
2. Create the ageing bucket calc from section 4 above.
3. **Chart 1 (bar):** `Ageing Bucket` on Columns, `SUM(Amount)` on Rows. Instantly shows ₹ stuck in 90+.
4. **Chart 2 (heatmap):** `Region` on Rows, `Ageing Bucket` on Columns, `SUM(Amount)` on Colour. Red = West's 90+ bucket is your problem child.
5. **KPI:** `Total Outstanding = SUM([Amount])` filtered to `Paid? = "No"` ; add `DSO ≈ (AR / Credit Sales) × 365` as a text tile.
6. Add a **Region filter** and a **Customer** quick filter. Combine all four onto one Dashboard sheet.

7. Publish to Tableau Cloud, schedule daily refresh, share to the collections team's email group only.

Outcome: the collections head opens one URL, filters to "West / 90+", and sees ₹8.75L from Tata Steel is the single biggest overdue — a decision that used to need a half-day Excel pull.

How it's tested

Interviews mix concept questions with a **live build test**.

Typical questions: - "When would you use Tableau over an Excel PivotTable?" (Answer: recurring, multi-user, needs scheduled refresh and interactivity; Excel for one-off analysis or heavy ad-hoc modelling.) - "Difference between a filter and a parameter?" (Filter limits data shown; parameter is a user-input value you plug into calcs/what-ifs.) - "What's an LOD expression / why FIXED?" (Compute at a grain independent of the view.) - "Bar vs pie — when?" (Almost always bar; pie only for 2–3 slices summing to 100%.) - "Star schema vs one flat table — why care?" (Fact + dimension tables; faster, cleaner measures, avoids duplication.)

The practical test (60–90 min, take-home or live): "Here's a messy sales/AR CSV. Build a dashboard showing [trend + breakdown + one KPI], make it filterable by region, and tell us the top insight." They score: correct chart choice, working calculated field, interactivity, cleanliness (no chartjunk), and whether you *found the insight*, not just drew boxes.

Common mistakes & how pros avoid them

- **Pie charts and 3D everything.** Pros default to bars and lines; colour carries one meaning, not five.
- **Building on a flat, un-modelled table.** Learn a simple star schema (one fact table, dimension tables for Date/Region/Customer). Makes measures correct and fast.
- **Over-engineering.** If two people need it once, it's an Excel PivotTable, not a Tableau server workbook. Pros match tool to lifespan.
- **No refresh plan.** A dashboard nobody scheduled dies in month two. Set the schedule *before* you share.
- **Sharing to "everyone."** Finance data is sensitive; set row/folder-level permissions.
- **Hard-coding dates/values in calcs.** Use `TODAY()`, parameters, and relative-date filters so it survives next month.
- **Ignoring performance.** Extracts (not live) for large data; filter at source; don't drag 2M rows into Tableau Public.

Learn-it roadmap & resources

Realistic time-to-proficiency: **3–5 weeks** part-time to job-ready if you already know Excel PivotTables.

Week	Focus
1	Excel PivotTables + charts mastery (the foundation)
2	Tableau Public: connect, dimensions/measures, basic charts
3	Calculated fields, LOD, filters, parameters, dashboards
4	Publishing, refresh, permissions; build 2 portfolio dashboards
5	Learn Power BI + DAX basics so you're tool-agnostic

Resources: - **Tableau Public** — free desktop tool + free portfolio hosting. Build here and put the link on your CV. - Tableau's official free training videos + "Makeover Monday" community datasets for practice. - **Power BI Desktop** — free download; the Microsoft Learn "PL-300" path is the best free curriculum. - Certifications: **Tableau Desktop Specialist** (~US$100) or **Microsoft PL-300 (Power BI Data Analyst)** — PL-300 is more recognised in Indian corporates because most run Microsoft 365.

Pick Power BI first if you're India-targeting and cost-sensitive: it's cheaper for employers, bundled with Office 365, and dominates Indian mid-market finance teams. Add Tableau for MNCs and analytics-heavy shops.

Quick-reference

Need	Excel	Tableau	Power BI
One-off analysis	✅ Best	Overkill	Overkill
Recurring multi-user dashboard	Weak	✅	✅
Scheduled auto-refresh	No	✅	✅
Cost for employer	Owned	₹₹₹	₹ (in O365)
Language for calcs	Formulas	Calc fields	DAX
Ad-hoc modelling / what-if	✅ Best	Weak	Medium

Chart cheat-sheet: trend → line · comparison → bar · part-to-whole → stacked bar (not pie) · variance → waterfall · relationship → scatter · distribution → histogram · concentration → heatmap.

Tableau essentials: Dimensions = "by what", Measures = "how much" · Filter = limit data · Parameter = user input · `{FIXED [X] : SUM([Y])}` = LOD · Extract = fast snapshot, Live = real-time.

Publish flow: clean model → build sheets → assemble dashboard → set filters → publish to Cloud/Service → schedule refresh → set permissions → share URL.

Decision rule: *Will 3+ people open this repeatedly?* → BI tool. *Just me, just now?* → Excel PivotTable.

Mini-project: An End-to-End Finance MIS Dashboard

What it is & where it's used

An **MIS (Management Information System) dashboard** is the one-page monthly picture that finance hands to a CEO/CFO/promoter: revenue vs target, gross margin, cash position, receivables ageing, top customers, expense heads, and where the business is bleeding. It sits on top of raw General Ledger (GL) exports and sales registers and turns thousands of transaction rows into 8-10 numbers a decision-maker actually reads.

This is the single most common "real" deliverable in Indian finance jobs. It's owned by **MIS Analysts, FP&A Analysts, Business Finance/Finance Business Partners, Financial Analysts, and Accounts Managers** in every SME, startup, and mid-cap. If you can build one unaided, you are immediately useful on day one — most freshers cannot.

The chapter builds the **full pipeline**: raw GL + sales CSV → a clean data model (star schema) → a Power BI / Excel dashboard → a monthly-reproducible process. Do it once here and you own a portfolio piece.

The gap: why companies want this (and college didn't teach it)

College teaches you to *read* a P&L and Balance Sheet that someone else prepared. Industry needs you to **manufacture** the P&L from a messy 40,000-row Tally/SAP dump — every month, on the 3rd working day, with zero errors, and explain the variances.

The specific gaps this closes:

College taught	Job needs
Interpret a finished P&L	Build P&L from a raw GL trial-balance export
Ratios in isolation	Ratios trended month-on-month with drill-down
"Prepare a statement" (once)	A *repeatable* process that runs every month unattended
One clean textbook dataset	Reconciling GL total to sales register, catching duplicates
Static answers	A live dashboard the CFO can filter by branch/product/month

The unstated skill is **data plumbing**: mapping a ledger's chart-of-accounts to reporting heads, joining sales to a calendar, and building it so next month is a one-click refresh, not a rebuild.

What "proficient" looks like

A job-ready person can, given a raw GL CSV and a sales register, unaided:

- Build a **cost-centre/account-head mapping table** so GL accounts roll up into P&L reporting lines.
- Construct a **star schema**: fact tables (GL, Sales) + dimension tables (Calendar, Account, Customer, Product).
- Write measures that produce **Revenue, COGS, Gross Margin %, EBITDA, MoM and YoY growth, MTD/YTD** correctly.
- Produce a **receivables ageing** bucket (0-30/31-60/61-90/90+).

- Tie the dashboard revenue back to the GL to the rupee (reconciliation).
- Refresh next month by **dropping in a new file** — no formula rewrites.
- Explain the top 3 variances in plain English.

Hands-on: how to actually do it

Raw inputs. Two files exported from Tally/ERP:

`gl.csv` — `Date, Voucher, AccountCode, AccountName, CostCentre, Debit, Credit` `sales.csv` — `Date, Invoice, CustomerCode, Customer, ProductCode, Product, Qty, Amount, GSTAmount`

And a mapping file you build by hand: `map_accounts.csv` — `AccountCode, ReportLine, ReportGroup` (e.g. `4001 → Sales → Revenue`, `5001 → Raw Material → COGS`, `6002 → Salaries → OpEx`).

Step 1 — Clean & shape in Python (repeatable)

```
import pandas as pd

gl   = pd.read_csv("gl.csv", parse_dates=["Date"])
mp   = pd.read_csv("map_accounts.csv")
sal  = pd.read_csv("sales.csv", parse_dates=["Date"])

# GL: signed amount (Dr +, Cr −) then map to report lines
gl["Amount"] = gl["Debit"].fillna(0) - gl["Credit"].fillna(0)
gl = gl.merge(mp, on="AccountCode", how="left")

# Catch unmapped accounts BEFORE they silently drop from the P&L
missing = gl[gl["ReportLine"].isna()]["AccountName"].unique()
assert len(missing) == 0, f"Unmapped accounts: {missing}"

gl["Month"] = gl["Date"].dt.to_period("M").astype(str)

# Monthly P&L pivot (income lines are credit-heavy so flip sign for reporting)
pnl = gl.groupby(["Month","ReportGroup","ReportLine"])["Amount"].sum().reset_index()
pnl.to_csv("fact_gl.csv", index=False)
sal.to_csv("fact_sales.csv", index=False)
```

Step 2 — Reconcile (never skip this)

```
gl_sales   = -gl.loc[gl["ReportLine"]=="Sales","Amount"].sum()   # credit → positive
reg_sales  = sal["Amount"].sum()
print("GL:", gl_sales, "Register:", reg_sales, "Diff:", gl_sales - reg_sales)
```

If the diff isn't zero (or a known GST/rounding gap), stop and investigate before dashboarding.

Step 3 — SQL alternative for the same rollup

```
SELECT  strftime('%Y-%m', g.Date)          AS Month,
        m.ReportGroup,
        m.ReportLine,
        SUM(g.Debit - g.Credit)            AS Amount
FROM    gl g
JOIN    map_accounts m ON g.AccountCode = m.AccountCode
GROUP BY Month, m.ReportGroup, m.ReportLine
ORDER BY Month;
```

Receivables ageing straight from open invoices:

```
SELECT CASE
         WHEN julianday('now') - julianday(Date) ≤ 30  THEN '0-30'
         WHEN julianday('now') - julianday(Date) ≤ 60  THEN '31-60'
         WHEN julianday('now') - julianday(Date) ≤ 90  THEN '61-90'
         ELSE '90+' END                    AS Bucket,
       SUM(Amount)                         AS Outstanding
FROM sales WHERE Paid = 0 GROUP BY Bucket;
```

Step 4 — Model it in Power BI (star schema)

Load `fact_gl`, `fact_sales`, and dimensions. Build a **Calendar** table and DAX measures:

```
Revenue     = CALCULATE( -SUM(fact_gl[Amount]), fact_gl[ReportLine]="Sales" )
COGS        = CALCULATE(  SUM(fact_gl[Amount]), fact_gl[ReportGroup]="COGS" )
Gross Margin   = [Revenue] - [COGS]
Gross Margin % = DIVIDE([Gross Margin], [Revenue])

Revenue LM  = CALCULATE([Revenue], DATEADD(Calendar[Date], -1, MONTH))
MoM %       = DIVIDE([Revenue] - [Revenue LM], [Revenue LM])
Revenue YTD = TOTALYTD([Revenue], Calendar[Date])
```

Step 5 — The Excel-only version (no Power BI)

Same result with `SUMIFS` off a helper table:

```
=SUMIFS(fact_gl[Amount], fact_gl[ReportLine],"Sales", fact_gl[Month],$B$1)*-1
=XLOOKUP([@AccountCode], map[AccountCode], map[ReportLine], "UNMAPPED")
=LET(rev, B5, cogs, B6, (rev-cogs)/rev)          // Gross Margin %
```

Use a PivotTable on `fact_gl` (Rows = ReportGroup/ReportLine, Columns = Month, Values = Sum of Amount) as the P&L engine, then a clean dashboard sheet referencing it with `GETPIVOTDATA`.

Worked example / mini-project

Fictional co: Surya Traders Pvt Ltd, April 2026. Raw GL rolls up to:

ReportGroup	ReportLine	Apr (₹)
Revenue	Sales	82,50,000
COGS	Raw Material	44,10,000
COGS	Freight Inward	3,20,000
OpEx	Salaries	11,40,000
OpEx	Rent	2,50,000
OpEx	Power & Fuel	1,80,000
Other	Interest	1,95,000

Derived dashboard KPIs:

Metric	Formula	Value
Revenue	—	₹82,50,000
COGS	44.10L + 3.20L	₹47,30,000
Gross Margin	Rev − COGS	₹35,20,000
Gross Margin %	35.20 / 82.50	**42.7%**
OpEx	11.40+2.50+1.80	₹15,70,000
EBITDA	GM − OpEx	₹19,50,000
EBITDA %	19.50 / 82.50	**23.6%**
PBT	EBITDA − Interest	₹17,55,000

Reconciliation: sales register total = ₹82,50,000 → matches GL Sales line to the rupee. Green tick.

Receivables ageing (from sales register, unpaid):

Bucket	₹	%
0-30	9,80,000	55%
31-60	4,60,000	26%
61-90	2,10,000	12%
90+	1,30,000	7%

MoM: March revenue was ₹78,00,000 → MoM = (82.50−78.00)/78.00 = **+5.8%**. Dashboard shows the KPI card in green with the arrow.

The one-page dashboard then has: 4 KPI cards (Revenue, GM%, EBITDA, PBT) with MoM arrows, a 12-month revenue+margin trend line, a top-10-customers bar, an expense-head donut, and the ageing bar. Filters:

Month, CostCentre. Next month: replace `gl.csv` and `sales.csv`, rerun the script / hit Refresh — done.

How it's tested

Practical assessment (the real gate): You're handed a raw GL CSV and told *"Build me a monthly P&L and a gross-margin trend by 5 pm."* They watch whether you (a) map accounts, (b) reconcile, (c) get the sign convention right, (d) make it refreshable.

Common interview questions: - "Walk me through how you'd build an MIS from a Tally export." (They want: export → map → model → reconcile → visualize → refresh.) - "Your dashboard revenue is ₹2L higher than the sales register. What do you check first?" (Duplicate voucher, credit note not netted, GST mixed into amount, unmapped account.) - "Difference between MTD and YTD, and how you'd compute both." - "How do you make this reproducible next month without rebuilding?" - "What's the difference between gross margin and EBITDA margin?"

Timed Excel/SQL screen: SUMIFS across a criteria set, XLOOKUP with a fallback, a GROUP BY rollup, an ageing CASE statement. 30-45 minutes.

Common mistakes & how pros avoid them

Mistake	Fix
Unmapped GL accounts silently dropped from P&L	`assert` / `XLOOKUP( ... ,"UNMAPPED")` check every refresh
Wrong Dr/Cr sign → income shows negative	Fix sign convention once in the model, flip income for display
GST amount counted as revenue	Keep taxable value and GST in separate columns; report net
Hard-coded month → breaks next cycle	Use a Calendar dimension + slicer, never type "April" in formulas
No reconciliation → confident but wrong numbers	Always tie dashboard revenue to the GL and sales register
Rebuilding from scratch monthly	Parameterise the file path; make refresh one click
Averaging percentages (avg of GM%s ≠ total GM%)	Compute ratios from summed numerator/denominator (DIVIDE)
Credit notes / returns ignored	Net them before totalling revenue

Learn-it roadmap & resources

Time to proficiency: ~3-4 weeks of focused evenings if you already know basic Excel.

- **Week 1** — Excel: SUMIFS, XLOOKUP, PivotTables, Power Query (get-and-transform). Build the P&L from CSV.
- **Week 2** — Data modelling: star schema, Calendar tables, DAX basics. Rebuild in Power BI (free desktop).
- **Week 3** — SQL: SELECT/JOIN/GROUP BY/CASE. Redo the rollup in SQLite/DB Browser (free).
- **Week 4** — Polish: reconciliation, ageing, variance commentary; write a one-page README so it's reproducible.

Resources: Power BI Desktop (free); DB Browser for SQLite (free); Microsoft Learn "Power BI" path (free); ICAI/company Tally exports for realistic data. **Certifications that carry weight:** Microsoft **PL-300 (Power BI Data Analyst)** and any structured SQL course. Build one real dashboard end-to-end and put the `.pbix` + screenshots in a portfolio — that beats any certificate in interviews.

Quick-reference

Need	Tool / snippet
Map account → report line	`=XLOOKUP(code, map[Code], map[Line], "UNMAPPED")`
Monthly P&L rollup	`SUMIFS(Amount, Line, "Sales", Month, B1)`
SQL rollup	`SUM(Debit-Credit) ... GROUP BY Month, ReportLine`
Revenue (DAX)	`CALCULATE(-SUM(Amount), Line="Sales")`
Gross Margin %	`DIVIDE([Revenue]-[COGS],[Revenue])`
MoM %	`DIVIDE([Rev]-[Rev LM],[Rev LM])`
YTD	`TOTALYTD([Revenue], Calendar[Date])`
Ageing bucket	`CASE WHEN days≤30 THEN '0-30' ...`
Reconcile	GL Sales total == sales register total == dashboard Revenue

Golden rule: map → model → **reconcile** → visualise → make it one-click refreshable. If it doesn't tie to the rupee, it isn't done.

AI & automation in finance

What it is & where it's used

"AI & automation in finance" means using three overlapping tool families to do finance work faster and with fewer errors:

1. **LLMs / Copilots** — ChatGPT, Claude, Microsoft 365 Copilot, Gemini. You give plain-English instructions ("prompts") and get back formulas, SQL, draft emails, variance commentary, reconciliations logic, GST notice replies, DCF explanations.
2. **RPA (Robotic Process Automation)** — bots that click through screens and move data the way a human would: Power Automate, UiPath, or even Excel VBA / Office Scripts. Used for repetitive, rule-based tasks — downloading bank statements, keying invoices into Tally, pulling GSTR-2B every month.
3. **Built-in AI features** — Copilot inside Excel, Power BI Copilot (writes DAX and narratives), Tally's AI add-ons, GSTN's e-invoice/IRP automation.

Where it shows up by role:

Role	AI/automation use
Accounts / AP-AR executive	Auto-key invoices, 3-way match bots, draft vendor follow-up emails
GST / tax associate	Reconcile GSTR-2B vs purchase register, draft replies to notices, summarise circulars
FP&A / MIS analyst	Copilot writes DAX/formulas, generates variance commentary, builds monthly deck
Audit / internal audit	Sample selection, exception testing, summarise policies vs. transactions
Treasury / controller	Bank-recon bots, cash-flow forecasting prompts

The gap: why companies want this (and college didn't teach it)

Your MBA/CA syllabus teaches **concepts** — accounting standards, cost of capital, GST law. It does not teach that in a real month-end you will re-key the same numbers 40 times, or that a junior who can make a bot download 30 bank statements in 3 minutes is worth 5x one who does it by hand.

The specific gap:

- College assumes **one clean dataset**. Industry gives you a messy Tally export, a bank PDF, and a GSTR-2B JSON that must be joined — under time pressure.
- College grades **the right answer**. Employers pay for **the right answer produced repeatably and fast**, with an audit trail.
- Nobody teaches **how to instruct an AI safely** — what to paste, what NOT to paste (client PAN, salary data), and how to verify AI output before it hits a filing.

The finance-specific risk employers fear: an analyst who blindly trusts a hallucinated number in a board deck, or pastes confidential data into a public chatbot. The person who understands **both the tool and its limits** is the one who gets hired and promoted.

What "proficient" looks like

A job-ready person can, unaided:

- Write a **prompt that produces a working Excel formula, SQL query, or DAX measure** on the first or second try, then verify it against a known-good number.
- Use Copilot in Excel/Power BI to **draft variance commentary**, then edit it for accuracy (not ship it raw).
- Build a **Power Automate flow** that moves files/data on a schedule without manual clicks.
- Record an **Excel macro / Office Script** for a repetitive cleanup and re-run it monthly.
- Know **data-governance red lines**: never paste PII, client-identifiable, or price-sensitive data into a public LLM; use the company-approved enterprise tool.
- **Sanity-check every AI output** — recompute totals, tie to source, flag anything the model "confidently" made up.

The bar is not "can code an AI model." It is "can use AI as a fast, cheap junior analyst — and catch its mistakes."

Hands-on: how to actually do it

1. Prompt patterns that work for finance

Use this **structure** every time: *Role → Context → Task → Format → Constraints.*

```
Role: You are an FP&A analyst.
Context: I have a table with columns Month, Budget, Actual (INR lakhs).
Task: Write an Excel formula for the % variance in column D,
      and flag "REVIEW" if the unfavourable variance exceeds 10%.
Format: Give the formula only, then one line explaining it.
Constraints: Assume data starts in row 2. No macros.
```

Reusable prompt patterns:

Pattern	Use it for	Example opener
Explain-then-do	Learning while producing	"Explain what a 3-way match is, then give the SQL to find mismatches."
Formula generator	Excel/DAX/SQL	"Write an XLOOKUP that returns HSN code from Sheet2, exact match."
Reconciliation logic	GSTR-2B vs books	"Given two tables by GSTIN+invoice no, list rows in A not in B."
Draft & polish	Emails, commentary	"Draft a polite payment-reminder email; overdue Rs 2,45,000, 45 days."
Summarise-to-decision	Circulars, policies	"Summarise this GST circular into 5 action points for a trader."
Red-team / check	Catch your own errors	"Here's my DCF. What assumptions look aggressive?"

2. Getting real formulas from an LLM

Ask, then paste this into Excel:

```
=IFERROR(XLOOKUP(A2, Vendors[GSTIN], Vendors[VendorName], "Not found"), "Error")
```

Variance flag (from the prompt above):

```
=IF((C2-B2)/B2 < -0.10, "REVIEW", TEXT((C2-B2)/B2, "0.0%"))
```

3. SQL the LLM writes for you — GSTR-2B vs purchase register

```
-- Invoices in books but MISSING in 2B (ITC at risk)
SELECT pr.gstin, pr.invoice_no, pr.taxable_value, pr.igst
FROM purchase_register pr
LEFT JOIN gstr2b b
       ON pr.gstin = b.gstin AND pr.invoice_no = b.invoice_no
WHERE b.invoice_no IS NULL;
```

4. Python snippet for a repetitive reconciliation

```
import pandas as pd

books = pd.read_excel("purchase_register.xlsx")
b2b   = pd.read_excel("gstr2b.xlsx")

merged = books.merge(b2b, on=["gstin", "invoice_no"],
                     how="outer", indicator=True)

only_books = merged[merged["_merge"] == "left_only"]   # ITC at risk
only_2b    = merged[merged["_merge"] == "right_only"]   # vendor filed, you didn't book
print(f"Missing in 2B: {len(only_books)} | Missing in books: {len(only_2b)}")
only_books.to_excel("itc_at_risk.xlsx", index=False)
```

5. DAX from Power BI Copilot (verify before trusting)

```
Variance % =
DIVIDE(
    SUM('P&L'[Actual]) - SUM('P&L'[Budget]),
    SUM('P&L'[Budget])
)
```

6. RPA basics — Power Automate (no code)

A first flow every finance junior should build:

1. Go to **make.powerautomate.com** → **Create** → **Scheduled cloud flow**.
2. Trigger: run **1st of every month, 9 AM**.
3. Action: **Outlook** → **Get attachments** where subject contains "Bank Statement".
4. Action: **OneDrive** → **Create file** in `/Recon/{Month}/`.
5. Action: **Send me an email** "3 statements filed."

For desktop clicking (e.g., keying into Tally), use **Power Automate Desktop** → record clicks → replace the manual keying loop.

Worked example / mini-project

Goal: Automate the monthly ITC reconciliation for a small trading firm, "Sharma Traders."

Inputs (reproduce with dummy data):

gstin	invoice_no	taxable_value (Rs)	igst (Rs)	source
27ABCDE1234F1Z5	INV-101	1,00,000	18,000	books
29PQRST5678K2Z1	INV-102	50,000	9,000	books
27ABCDE1234F1Z5	INV-101	1,00,000	18,000	2B
24LMNOP9012Q3Z9	INV-777	75,000	13,500	2B

Step 1 — Prompt the LLM:

```
Role: GST analyst. I have purchase_register and gstr2b tables
(gstin, invoice_no, taxable_value, igst). Write Python to output:
(a) invoices in books not in 2B, (b) in 2B not in books,
(c) total IGST at risk. Save to Excel.
```

Step 2 — Run the Python from section 4. Result:

- INV-102 → in books, **not in 2B** → Rs 9,000 IGST at risk (chase vendor to file).
- INV-777 → in 2B, **not in books** → Rs 13,500 (book it or query the vendor).

Step 3 — Copilot drafts the commentary: "For June, Rs 9,000 ITC is blocked pending vendor filing of INV-102; one unrecorded inward supply (INV-777, Rs 13,500 IGST) requires booking."

Step 4 — You verify: recompute `SUMIF` on IGST, tie totals to Tally's GSTR-2 report, confirm the two exceptions by eye. Then file.

Step 5 — Automate next month: save the Python + a Power Automate flow that drops both files into a folder and emails you the exception count. Month-end task drops from ~2 hours to ~10 minutes.

How it's tested

Interview questions:

- "How would you use ChatGPT/Copilot in your day, and what would you *never* paste into it?" (They test data-governance awareness.)

- "The AI gives you a formula that returns a wrong total. What's your process?" (Verification mindset.)
- "Walk me through automating a task you did manually." (Do you think in repeatable processes?)
- "What are LLM hallucinations and why do they matter for a filing?"

Practical / assessment tests companies give:

- **Timed Copilot/Excel test:** "Here's a messy 5,000-row export. Use any AI help to reconcile and produce a variance summary in 30 minutes." They watch *how* you prompt and *whether* you verify.
- **Prompt-craft screen:** given a business ask, write the prompt(s) you'd use.
- **RPA task:** "Build a flow that emails a reminder when an invoice is >30 days overdue."
- **Judgment case:** "This AI-generated board number looks off — find the error." (The error is planted.)

Common mistakes & how pros avoid them

Mistake	How pros avoid it
Pasting client PAN/GSTIN/salary into a public chatbot	Use enterprise/approved tools; anonymise; paste structure, not sensitive values
Shipping AI output unchecked	Always recompute totals, tie to source, spot-check 3-5 rows
Trusting a hallucinated citation (fake section/circular no.)	Verify every legal reference against the actual GST portal / bare act
Vague prompts ("analyse this")	Use Role-Context-Task-Format-Constraints; give a sample of the data
Automating a broken process	Fix and document the manual process first, *then* automate
No audit trail	Keep the prompt, the source file, and a note of what you verified
Over-automating one-off tasks	Automate only recurring, rule-based work (the 80/20)

Learn-it roadmap & resources

Time to job-ready: ~4-6 weeks, part-time.

Week	Focus
1	Prompt patterns; use ChatGPT/Copilot to generate Excel formulas daily
2	LLM for SQL + reconciliation logic; learn to verify every output
3	Power Automate — build 2-3 scheduled/desktop flows
4	Copilot in Excel & Power BI (DAX + narratives)
5-6	End-to-end mini-project (the ITC recon above); learn data-governance rules

Resources:

- **Free:** Microsoft Learn — Power Automate & Copilot paths; Google's "Prompting Essentials"; Anthropic/OpenAI prompt guides; YouTube (Leila Gharani for Excel+AI).

- **Paid:** Microsoft 365 Copilot licence (if employer provides); UiPath Academy (free RPA certs); Coursera "Generative AI for Finance."
- **Certifications worth it:** Microsoft **PL-900** (Power Platform), **UiPath RPA Associate**, Microsoft **PL-300** (Power BI, includes Copilot).

Staying relevant: treat AI as a **force multiplier, not a replacement**. The finance jobs that survive belong to people who pair domain judgment (accounting standards, GST law, materiality) with tool fluency. Learn one new automation each month; keep a personal "prompt library" of what works.

Quick-reference

Prompt skeleton: `Role → Context (with data sample) → Task → Format → Constraints`

Key formulas / snippets:

Need	Snippet
Lookup with error handling	`=IFERROR(XLOOKUP(A2, tbl[key], tbl[val], "NA"), "Err")`
Variance flag	`=IF((Actual-Budget)/Budget < -0.1, "REVIEW", "OK")`
Missing in 2B (SQL)	`LEFT JOIN ... WHERE b.invoice_no IS NULL`
Recon (Python)	`books.merge(b2b, on=[...], how="outer", indicator=True)`
Variance % (DAX)	`DIVIDE(SUM(Actual)-SUM(Budget), SUM(Budget))`

Red lines — never paste into a public LLM: PAN, Aadhaar, GSTIN-linked client data, salary/HR data, price-sensitive/unpublished financials.

Verify-before-ship checklist: recompute totals · tie to source · spot-check 3-5 rows · verify every cited section/circular · keep prompt + source as audit trail.

Automate only if: recurring + rule-based + stable process. One-off or judgment-heavy → do it manually.

Data literacy & storytelling

What it is & where it's used

Data literacy is the ability to read, interpret, question, and communicate numbers so they change a decision. Storytelling is the packaging: turning a correct-but-inert analysis into a narrative a busy CFO, partner, or promoter acts on in 90 seconds.

This is the last-mile skill that separates an analyst from a "spreadsheet operator." You can build a perfect variance model, but if the deck buries the one number that matters under 14 pie charts, nobody moves. Every finance role touches this:

Role	Where storytelling shows up
FP&A analyst	Monthly MIS deck, budget-vs-actual review, board pack
Audit (CA firm)	Summary of audit findings, ICFR observations to audit committee
Tax / GST	Reconciliation summary (GSTR-2B vs books) for management sign-off
Investment/equity research	One-page thesis, target-price bridge
Controllership	Flash results, cash-flow story, working-capital review
Consulting / TP	Client-facing "so what" slides

The output is almost always one of three things: a **one-page executive summary**, a **single chart that carries an argument**, or a **verbal 60-second answer** in a review meeting.

The gap: why companies want this (and college didn't teach it)

College trains you to *produce* answers — compute NPV, pass journal entries, prepare a cash-flow statement. It grades completeness. Industry rewards *compression and consequence*: what do I do differently because of this number?

The specific gaps employers complain about:

- **No "so what."** Fresh hires present *what happened* ("revenue is ₹42 Cr") but not *why it matters* ("revenue is ₹42 Cr, 8% below plan, driven entirely by the North zone, and it puts the Q4 incentive pool at risk").
- **Data dump, not argument.** A 30-tab workbook emailed with "PFA" instead of a 3-bullet takeaway.
- **Chart illiteracy.** Pie charts with 11 slices, dual-axis charts that mislead, 3-D bars, rainbow colours with no meaning.
- **No audience calibration.** Same detail for the analyst and the board. Executives want the decision; they'll ask for detail.
- **Burying the lede.** The headline finding on slide 23 instead of slide 1.

A CA syllabus never grades "did the audit committee understand the risk in 2 minutes?" — but that is exactly the billable skill.

What "proficient" looks like

A job-ready person can, unaided:

1. Open any dataset and state the **top 3 findings in one sentence each**, ranked by decision impact.
2. Write a **BLUF (Bottom Line Up Front)** executive summary: conclusion first, evidence second.
3. Pick the **right chart for the question** without thinking — comparison → bar, trend → line, part-to-whole → stacked bar (rarely pie), correlation → scatter, distribution → histogram.
4. Build a **variance/bridge (waterfall) chart** that explains a movement from A to B.
5. Strip **chart-junk**: no gridline clutter, no 3-D, no redundant legends, direct labels, one accent colour to highlight the point.
6. Tailor the same analysis into three lengths: **60-second verbal, one-page, full appendix.**
7. Defend every number ("where did ₹8 Cr come from?") and state the **action** the number implies.

Hands-on: how to actually do it

1. The BLUF structure (memorise this)

```
[Conclusion + recommendation]  ← one sentence, put it FIRST
   Because [3 supporting facts, ranked]
   Which means [the decision / risk / rupee impact]
   I recommend [specific action + owner + date]
```

Bad: "Please find the sales analysis attached." Good: "**We will miss the ₹500 Cr annual target by ~₹30 Cr unless North zone recovers.** North is 18% below plan (₹22 Cr gap), driven by two lost distributors; West and South are on track. Recommend the CSO review North distributor terms by 15th."

2. Chart choice — decision table

Your question	Chart	Avoid
A vs B vs C (compare)	Horizontal **bar**, sorted	Pie
Change over time	**Line**	Area stacks
Part-to-whole (2–4 parts)	**Stacked bar / 100% bar**	Pie > 4 slices
What moved the total A→B	**Waterfall (bridge)**	Two side-by-side bars
Relationship of two metrics	**Scatter**	Dual-axis line
Actual vs target	**Bullet chart / bar + target line**	Gauge
Many categories, one metric	Sorted **bar**, top-N + "Others"	Rainbow columns

3. Excel: build a variance bridge (waterfall)

Native waterfall (Excel 2016+): select the data, **Insert** → **Charts** → **Waterfall**. Then double-click the total column → check **"Set as Total"**.

To compute the bridge components for Plan → Actual:

```
Start (Plan):        =Plan
Volume effect:       =(Act_Vol - Plan_Vol) * Plan_Price
Price effect:        =(Act_Price - Plan_Price) * Act_Vol
Mix / other:         =Actual - Plan - Volume_effect - Price_effect
End (Actual):        =Start + SUM(effects)   ← must reconcile
```

Highlight the worst driver in a contrasting colour (right-click that point → Fill → dark red). Everything else stays grey. **One accent colour = one message.**

4. Excel: turn a table into a headline

```
="Revenue "&TEXT([@Actual],"₹#,##0,,\ \C\r")&", "&
 TEXT(([@Actual]-[@Plan])/[@Plan],"0%")&
 IF([@Actual]<[@Plan]," below plan"," above plan")
```

This auto-writes: `Revenue ₹42 Cr, -8% below plan`. Put it as a dynamic title so the chart title *is* the takeaway, not "Chart 1".

5. Direct labelling instead of legends

Legends force the eye to ping-pong. In a line chart, delete the legend and add a text box at the end of each line with the series name in the line's colour. Fewer than 4 series → always direct-label.

6. Python: a clean, chart-junk-free plot

```
import matplotlib.pyplot as plt

zones  = ['North', 'West', 'South', 'East']
gap    = [-22, 3, 5, -1]          # ₹ Cr vs plan
colors = ['#c0392b' if g < -5 else '#bdc3c7' for g in gap]

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.barh(zones, gap, color=colors)
ax.axvline(0, color='#333', lw=0.8)
ax.set_title('North drags the plan by ₹22 Cr', loc='left', fontsize=13, weight='bold')
for s in ['top', 'right', 'bottom']:      # kill chart-junk
    ax.spines[s].set_visible(False)
ax.tick_params(length=0)
for y, g in enumerate(gap):               # direct data labels
    ax.text(g, y, f' {g:+d}', va='center',
            ha='left' if g > 0 else 'right')
ax.set_xticks([])                         # numbers on bars, no axis
plt.tight_layout(); plt.savefig('zone_gap.png', dpi=150)
```

The title states the conclusion; grey/red does the arguing; no gridlines, no legend, no axis noise.

7. Power BI / DAX: a KPI that tells the story

```
Rev vs Plan % =
VAR act  = SUM(Sales[Actual])
VAR plan = SUM(Sales[Plan])
RETURN DIVIDE(act - plan, plan)

Rev Status =
SWITCH(TRUE(),
    [Rev vs Plan %] < -0.05, "🔴 Behind",
    [Rev vs Plan %] <  0,    "🟡 Watch",
    "🟢 On track")
```

Drive conditional formatting off `Rev Status` so the reader sees red *before* they read a single number.

Worked example / mini-project

Scenario. You are FP&A at an Indian FMCG distributor. FY revenue = ₹470 Cr against a ₹500 Cr plan. The MD wants "why" in one page tomorrow.

Raw data (₹ Cr):

Zone	Plan	Actual	Gap
North	120	98	-22
West	150	153	+3
South	130	135	+5
East	100	84	-16
Total	**500**	**470**	**-30**

Step 1 — find the story. The ₹30 Cr miss is *not* broad — West and South beat plan. It's concentrated in North (-22) and East (-16), offset by +8 elsewhere. That's the lede.

Step 2 — one-line BLUF.

> **We closed FY at ₹470 Cr, ₹30 Cr (6%) short — but the miss is entirely North + East (-₹38 Cr); West & South over-delivered by ₹8 Cr.**

Step 3 — one chart. Horizontal bar of gap-by-zone, North & East in red, others grey, title = "Two zones caused the entire ₹30 Cr miss" (the Python snippet above renders exactly this).

Step 4 — the "so what" + action.

Finding	Rupee impact	Recommended action	Owner
North lost 2 distributors	-₹22 Cr	Renegotiate margins, re-appoint by Q1	Zonal Head N
East supply outages	-₹16 Cr	Add secondary depot	Supply Chain
West/South momentum	+₹8 Cr	Protect; replicate scheme	CSO

Step 5 — the one-pager layout:

```
┌──────────────────────────────────────────────┐
│ FY Revenue: ₹470 Cr  (−₹30 Cr / −6% vs plan)  │  ← BLUF banner
│                                              │
│ [ zone-gap bar chart, North & East in red ]  │  ← ONE chart
│                                              │
│ • 2 zones = 100% of miss                     │  ← 3 bullets
│ • West & South +₹8 Cr, on track              │
│ • Recovery plan → +₹25 Cr addressable in Q1  │
│                                              │
│ Ask: approve North distributor margin revision │  ← the decision
└──────────────────────────────────────────────┘
```

Full zone/SKU/month detail goes in the appendix — referenced, not shown. The MD reads the banner, glances at one chart, and approves an action. That is data storytelling.

How it's tested

Interviews and assessments target the last mile explicitly:

Verbal / whiteboard questions - "You have 30 seconds and the CFO's attention. Give me the headline of this P&L." (tests BLUF instinct) - "This chart is a pie with 9 slices — what's wrong and how would you fix it?" - "Revenue is up 5% and profit is down 10%. Tell me the story." (margin/mix reasoning + narrative) - "What's the one number on this dashboard you'd escalate?"

Practical assessments - **Timed deck test:** given a messy 5-tab workbook, produce a 1-slide summary in 45 minutes. Graded on: is the conclusion first? is there one clear chart? is there a recommended action? - **Chart critique:** they hand you a deliberately bad chart (3-D, dual-axis, rainbow) and ask you to redo it. - **Case presentation:** analyse quarterly results and present to a panel; they interrupt with "so what?" and "what would you do?" - **Email test:** "Write the covering email a manager would actually read" — tests compression.

Scoring is rarely about spreadsheet mechanics; it's *did the reader know the decision without asking a follow-up question.*

Common mistakes & how pros avoid them

Mistake	Fix pros use
Conclusion on the last slide	BLUF — decision on slide 1, banner, or subject line
Pie charts / 3-D / rainbow	Sorted bars; one accent colour on the point that matters
Dual-axis to fake a correlation	Two panels or a scatter; never trick the eye
Chart title says "Revenue by Zone"	Title = the takeaway: "North caused the miss"
Legend the reader must decode	Direct-label series at the line's end
Presenting data, not the "so what"	Every number followed by "…which means…"
Same detail for board & analyst	Layer: 1-line → 1-page → appendix
Precision theatre (₹4,72,38,914)	Round to ₹4.7 Cr — precision ≠ credibility
Truncated y-axis exaggerating change	Start bars at zero; note breaks explicitly
No recommended action	End with action + owner + date

The professional's tell: they can delete 80% of the deck and the message survives.

Learn-it roadmap & resources

Realistic time-to-proficiency: 4–8 weeks of deliberate practice on top of existing Excel/BI skills. It's a *rewiring of habits*, not a new tool.

Week	Focus
1	BLUF structure; rewrite 5 old emails/decks conclusion-first
2	Chart choice table; remake 10 bad charts you find online
3	Build 3 variance/waterfall bridges in Excel from real data
4	One-page executive summaries — do 5, get feedback
5–6	Power BI storytelling: KPI cards, conditional formatting, drill-through
7–8	Mock panel presentations; practise the "so what?" reflex

Resources - *Storytelling with Data* — Cole Nussbaumer Knaflic (the standard; free blog + monthly challenge at storytellingwithdata.com). - *The Pyramid Principle* — Barbara Minto (BLUF / top-down structuring). - *Show Me the Numbers* — Stephen Few (chart design, anti-chart-junk). - Free: Google "Data Studio / Looker Studio" tutorials; matplotlib + seaborn galleries; the SWD blog exercises. - Certification: **Microsoft PL-300 (Power BI Data Analyst)** — includes a reporting/visual-design section; strong signal for FP&A/analyst roles in India (₹ market rate for the exam ~US$165). - Practice reps: take any listed Indian company's quarterly investor presentation and rewrite one slide better.

Quick-reference

BLUF template: Conclusion → because (3 facts) → which means (₹ impact) → I recommend (action, owner, date).

Chart picker: compare → sorted bar · trend → line · part-to-whole → stacked/100% bar (not pie) · movement A→B → waterfall · relationship → scatter · vs target → bullet.

Anti-chart-junk checklist: ☐ conclusion in the title ☐ one accent colour ☐ no 3-D ☐ no gridline clutter ☐ direct labels, no legend if <4 series ☐ y-axis starts at zero ☐ rounded numbers ☐ ≤1 core chart per message.

The 3 layers: 60-sec verbal → 1-page summary → full appendix.

The one test that matters: *Can the reader state the decision without asking a follow-up?* If no, cut and re-lead.

Excel dynamic takeaway title:

```
=TEXT([@Actual],"₹#,##0,,\ \C\r")&" ("&TEXT(([@Actual]-[@Plan])/[@Plan],"+0%;-0%")&" vs plan)"
```

Rounding rule: report to 2 significant figures for narrative (₹4.7 Cr), keep full precision in the appendix.

The data/SQL interview & test

What it is & where it's used

The SQL interview is the single most common practical filter between you and a data-touching finance job. Once a job description says "SQL", "data extraction", "pull your own numbers", or "self-serve analytics", assume there is a live or take-home SQL test — an MBA and a CA Intermediate do not exempt you from it.

Where it shows up:

Role	What the SQL screen looks like
FP&A / Finance analyst	3–5 questions on a sales/GL table, timed 30–45 min
Financial / Business analyst (fintech, e-comm)	Live shared editor, 45–60 min, joins + window functions
Revenue / Billing analyst	Take-home: reconcile invoices vs. payments
Data analyst (finance vertical)	HackerRank/Stratascratch style, 60–90 min
Audit analytics / Internal audit	Case: find duplicate vendors, split POs

The bar is not "can you write a `SELECT *`". It is "can you turn a vague business question into a correct query, unaided, under time pressure, and explain it."

The gap: why companies want this (and college didn't teach it)

College teaches you to *read* financial statements someone else prepared. The job is to *produce* the numbers from raw transactional tables — and those tables have nulls, duplicates, wrong data types, and 40 million rows. An MBA case study hands you a clean Exhibit 4; the real ERP hands you `sales_line_item` with `amount` stored as text and `INV-0001` sometimes typed `inv 0001`.

The specific gaps a SQL screen exposes:

- **Aggregation logic.** You know GROUP BY conceptually, but freeze on "revenue per customer *for their first month only*."
- **Joins under ambiguity.** LEFT vs INNER changes your revenue number. Getting this wrong = a wrong board deck.
- **Window functions.** Running totals, month-over-month growth, "top 3 per region", rank — never covered in a finance degree, asked in ~60% of screens.
- **Data hygiene instinct.** Pros check for duplicates and nulls *before* trusting a total. Freshers report the first number they get.

Companies test SQL because it is the cheapest reliable proxy for "can this person get the right number without hand-holding."

What "proficient" looks like

The concrete, job-ready bar. You can, unaided:

1. Filter and aggregate: `WHERE`, `GROUP BY`, `HAVING`, `COUNT/SUM/AVG`.

2. Join 2–4 tables and *know* why you chose INNER vs LEFT.
3. Use `CASE WHEN` for bucketing and conditional sums.
4. Write window functions: `ROW_NUMBER`, `RANK`, `SUM() OVER`, `LAG/LEAD`.
5. Structure a query with CTEs (`WITH`) instead of nested subqueries.
6. Handle nulls (`COALESCE`), dedupe, and cast types.
7. Compute finance staples: MoM growth, running total, top-N-per-group, cohort/retention, churn.
8. Read a business question and translate it — the actual skill being tested.

If you can do 1–6 fluently and reason through 7–8 out loud, you pass most finance-adjacent screens.

Hands-on: how to actually do it

Assume two tables:

```
customers(customer_id, name, city, signup_date)
orders(order_id, customer_id, order_date, amount, status)
```

Filter + aggregate — revenue per city, paid orders only:

```
SELECT c.city,
       SUM(o.amount)      AS total_revenue,
       COUNT(*)           AS num_orders
FROM   orders o
JOIN   customers c ON c.customer_id = o.customer_id
WHERE  o.status = 'PAID'
GROUP BY c.city
HAVING SUM(o.amount) > 100000
ORDER BY total_revenue DESC;
```

Conditional aggregation (pivot without a pivot) — paid vs refunded per month:

```
SELECT DATE_TRUNC('month', order_date) AS mth,
       SUM(CASE WHEN status='PAID'     THEN amount ELSE 0 END) AS paid,
       SUM(CASE WHEN status='REFUNDED' THEN amount ELSE 0 END) AS refunded
FROM   orders
GROUP BY 1
ORDER BY 1;
```

LEFT JOIN to find the gap — customers who never ordered:

```
SELECT c.customer_id, c.name
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;
```

Window function — running total and month-over-month growth:

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS mth,
         SUM(amount) AS revenue
  FROM   orders
  WHERE  status = 'PAID'
  GROUP BY 1
)
SELECT mth,
       revenue,
       SUM(revenue) OVER (ORDER BY mth)                 AS running_total,
       revenue - LAG(revenue) OVER (ORDER BY mth)       AS mom_change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY mth))
             / LAG(revenue) OVER (ORDER BY mth), 1)      AS mom_pct
FROM   monthly
ORDER BY mth;
```

Top-N per group — top 2 customers by spend in each city:

```
WITH ranked AS (
  SELECT c.city, c.name,
         SUM(o.amount) AS spend,
         ROW_NUMBER() OVER (PARTITION BY c.city
                            ORDER BY SUM(o.amount) DESC) AS rn
  FROM   orders o
  JOIN   customers c ON c.customer_id = o.customer_id
  GROUP BY c.city, c.name
)
SELECT city, name, spend FROM ranked WHERE rn ≤ 2;
```

Data hygiene — the check pros run first:

```
-- duplicate orders?
SELECT order_id, COUNT(*) FROM orders GROUP BY order_id HAVING COUNT(*) > 1;
-- nulls hiding in the money column?
SELECT COUNT(*) FROM orders WHERE amount IS NULL;
```

Worked example / mini-project

Case (India e-commerce, reproduce this locally): "Give me GST-inclusive net revenue by month for FY2024-25, the MoM growth, and flag any month where refunds exceeded 5% of gross."

Seed data (paste into SQLite / Postgres):

```
CREATE TABLE orders(order_id INT, customer_id INT, order_date DATE,
                    amount NUMERIC, status TEXT);
INSERT INTO orders VALUES
(1,101,'2024-04-05', 50000,'PAID'),
(2,102,'2024-04-18', 30000,'PAID'),
(3,103,'2024-04-22', 12000,'REFUNDED'),
(4,101,'2024-05-10', 80000,'PAID'),
(5,104,'2024-05-15', 20000,'PAID'),
(6,105,'2024-05-27', 60000,'REFUNDED'),
(7,102,'2024-06-02', 90000,'PAID'),
(8,103,'2024-06-19',  5000,'REFUNDED');
```

Amounts are GST-inclusive at 18%. Solution:

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS mth,
         SUM(CASE WHEN status='PAID' THEN amount ELSE 0 END)     AS gross_paid,
         SUM(CASE WHEN status='REFUNDED' THEN amount ELSE 0 END) AS refunds
  FROM orders
  GROUP BY 1
)
SELECT mth,
       gross_paid,
       ROUND(gross_paid / 1.18, 0)                        AS net_ex_gst,
       ROUND(gross_paid - gross_paid/1.18, 0)             AS gst_component,
       ROUND(gross_paid - LAG(gross_paid) OVER (ORDER BY mth), 0) AS mom_change,
       CASE WHEN refunds > 0.05 * (gross_paid + refunds)
            THEN 'FLAG' ELSE 'OK' END                     AS refund_flag
FROM m
ORDER BY mth;
```

Expected reading: April gross ₹80,000 (net ₹67,797, GST ₹12,203), May ₹100,000 (MoM +₹20,000) but refunds ₹60,000 vs gross ₹100,000 → **FLAG** (>5%). Being able to *narrate* that last line — "May looks great on revenue but refund rate is 37%, so I'd investigate before reporting growth" — is what gets you the offer.

How it's tested

Format you'll actually face:

Type	Detail
Live shared editor	Zoom + DB Fiddle / CoderPad; 3–4 questions, 45 min, they watch you think
Timed platform	HackerRank / StrataScratch / DataLemur; auto-graded, 60–90 min
Take-home	A CSV + "answer these 5 business questions", 24–48 hr
Verbal-only	Whiteboard/no-run; they judge structure, not exact syntax

Questions that recur:

- "Second-highest salary / order value." (Tests window functions or subquery.)
- "Customers who ordered in Jan but not Feb." (Anti-join / `NOT IN` / `EXCEPT`.)
- "Difference between `WHERE` and `HAVING`; `INNER` vs `LEFT JOIN`." (Verbal.)
- "MoM revenue growth %." (Window + `LAG`.)
- "Top 3 products per category." (`ROW_NUMBER` + `PARTITION BY`.)
- "Deduplicate this table keeping the latest row." (`ROW_NUMBER()` then `WHERE rn=1`.)
- "How would you check this number is right?" (The hygiene answer — most freshers miss it.)

How they grade: correctness first, then whether you clarified assumptions, then readability (CTEs > nested subqueries), then that you sanity-checked the output.

Common mistakes & how pros avoid them

Mistake	Fix
Diving into SQL before understanding the question	Restate it: "So 'active customer' means ordered in last 30 days?" — always clarify first
Using `INNER JOIN` when the question needs unmatched rows	If the ask is "who *didn't*", it's a `LEFT JOIN … IS NULL` or `EXCEPT`
`COUNT(*)` vs `COUNT(column)` confusion	`COUNT(col)` ignores nulls; know which you mean
Filtering on an aggregate in `WHERE`	Aggregate conditions go in `HAVING`, not `WHERE`
Nulls silently breaking sums/joins	`COALESCE(amount,0)`; check `IS NULL` early
Wrong grain → double counting after a join	Aggregate before joining, or check for fan-out with a row count
Reporting the first number without checking	Run the dedupe + null check first; state "I verified no dupes"
Nested subquery spaghetti	Use `WITH` CTEs — readable and graders reward it
Integer division (`1/2 = 0`)	Multiply by `100.0` or cast for percentages

Learn-it roadmap & resources

Realistic time-to-proficiency from zero, part-time: **4–6 weeks** to pass most finance screens.

Week	Focus
1	SELECT/WHERE/GROUP BY/ORDER BY/HAVING — SQLBolt (free, ~4 hrs)
2	Joins (all types), CASE, subqueries — Mode SQL Tutorial (free)
3	Window functions + CTEs — the part that actually differentiates you
4	Drill interview questions — DataLemur (free tier), StrataScratch
5–6	Timed mocks + one take-home; build the mini-project above in SQLite

Resources:

- **Free:** SQLBolt, Mode Analytics SQL Tutorial, DataLemur (Nick Singh, finance-heavy), PostgreSQL Tutorial, LeetCode Database (50 problems).
- **Practice envs:** DB Fiddle, sqliteonline.com — zero install, paste and run.
- **Paid (optional):** StrataScratch (~$30/mo, real company questions), "Ace the Data Science Interview" book.
- **Certification:** not required and rarely asked for in finance roles — a GitHub repo with 3 solved case queries beats a certificate. If you want one, Google Data Analytics (Coursera) or Microsoft DP-900 signal basics.

Install SQLite locally (`sqlite3`, ships free) and reproduce every query above — running beats reading.

Quick-reference

```
-- Skeleton
SELECT col, AGG(x) FROM t
JOIN t2 ON t.id=t2.id
WHERE  filter_rows
GROUP BY col
HAVING AGG(x) > n            -- filter on aggregate
ORDER BY col LIMIT k;

-- Window
ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)  -- top-N per group
SUM(x)     OVER (ORDER BY d)                        -- running total
LAG(x)     OVER (ORDER BY d)                        -- prev row → MoM
RANK() / DENSE_RANK()                               -- ties handling

-- Patterns
COALESCE(x,0)                       -- null-safe
LEFT JOIN … WHERE b.id IS NULL      -- anti-join ("didn't")
A EXCEPT B                          -- rows in A not in B
100.0 * a / b                       -- avoid integer division
WITH cte AS (…) SELECT … FROM cte   -- readable > nested
```

Question type	Weapon
Top-N per group	`ROW_NUMBER() … PARTITION BY`
MoM / running total	`LAG` / `SUM() OVER`
"Who didn't…"	LEFT JOIN … IS NULL / `EXCEPT`
Nth highest	window or correlated subquery
Pivot	`SUM(CASE WHEN …)`
Dedupe	`ROW_NUMBER()`, keep `rn=1`

Golden rule: clarify the question → check the data → write the CTE → sanity-check the number → explain it out loud.

Cheat-sheet: SQL, DAX & pandas

What it is & where it's used

This chapter is a dense, printable reference for the three languages that do the heavy lifting in a modern finance-analytics job: **SQL** (pull and shape data from a database), **DAX** (build measures inside Power BI / Excel Power Pivot), and **pandas** (Python data-wrangling). You use SQL to answer "how much did we bill client X in FY25?" against a warehouse; DAX to make a P&L dashboard that recalculates as a CFO clicks filters; pandas to reconcile a 200k-row bank statement against the ledger in 30 seconds.

Roles that touch these daily: **FP&A analyst**, **finance/business analyst**, **management-reporting (MIS) executive**, **revenue/GL accountant in a shared-service centre**, **credit / risk analyst**, and **audit-analytics** teams at the Big 4. In Indian GCCs (Deloitte USI, EY GDS, Accenture, Genpact, TresVista) a job ad that says "hands-on with Power BI and SQL" is testing exactly the muscle memory below.

The gap: why companies want this (and college didn't teach it)

An MBA-Finance or CA course teaches you *what* a P&L, a reconciliation, or a variance is. It does not teach you to compute it over 500,000 rows without Excel choking. The gap is **mechanical fluency at scale**:

- College: `VLOOKUP` two sheets. Job: `LEFT JOIN` a 4-million-row invoice table to a customer master and not create duplicates.
- College: a static ratio in a template. Job: a `CALCULATE(SUM( … ), FILTER( … ))` measure that stays correct when the user slices by region *and* month.
- College: retype numbers. Job: a pandas script that runs the same reconciliation every morning, unattended.

Employers pay for the person who can turn a vague ask ("why did margin drop in Q3?") into a query, a measure and a chart in an hour. That is a skill nobody grades in an exam, so it screens candidates hard.

What "proficient" looks like

The concrete bar an interviewer expects you to clear **unaided**:

- **SQL**: write a `JOIN` + `GROUP BY` + `HAVING` query; use a window function (`SUM() OVER` , `ROW_NUMBER()`) for running totals and de-duplication; explain the difference between `WHERE` and `HAVING` , and `INNER` vs `LEFT` join, without notes.
- **DAX**: know that a measure recomputes in *filter context*; write `CALCULATE` with a filter; build a YoY / YTD measure using `SAMEPERIODLASTYEAR` and `TOTALYTD` ; understand `SUM` (a column) vs `SUMX` (row-by-row).
- **pandas**: load a CSV/Excel, `merge` two frames, `groupby().agg()` , `pivot_table` , handle NaNs, and export back to Excel — cleanly, in under 20 lines.

Hands-on: how to actually do it

SQL — the clauses you use every day

Logical execution order (memorise this — it explains 90% of bugs): `FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT`.

```
-- Revenue by customer for FY25 (Apr-24 to Mar-25), > 10 lakh only
SELECT c.customer_name,
       SUM(i.amount)                      AS total_rev,
       COUNT(*)                           AS invoice_count
FROM   invoices i
JOIN   customers c ON c.customer_id = i.customer_id   -- INNER: only matched rows
WHERE  i.invoice_date BETWEEN '2024-04-01' AND '2025-03-31'
GROUP  BY c.customer_name
HAVING SUM(i.amount) > 1000000             -- filter AFTER aggregation
ORDER  BY total_rev DESC
LIMIT  20;
```

```
-- Window functions: running total + latest row per customer
SELECT customer_id, invoice_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY invoice_date) AS running_total,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY invoice_date DESC) AS rn
FROM   invoices;          -- keep WHERE rn = 1 in an outer query to get the latest invoice
```

```
-- Categorise ageing (AR buckets) with CASE
SELECT invoice_id, amount,
       CASE WHEN DATEDIFF(CURRENT_DATE, due_date) ≤ 0   THEN 'Not due'
            WHEN DATEDIFF(CURRENT_DATE, due_date) ≤ 30  THEN '0-30'
            WHEN DATEDIFF(CURRENT_DATE, due_date) ≤ 90  THEN '31-90'
            ELSE '90+' END AS ageing_bucket
FROM   invoices WHERE status = 'OPEN';
```

`COALESCE(col, 0)` swaps NULLs for 0; `LEFT JOIN` keeps unmatched left rows (use it to find invoices with **no** payment: `WHERE p.payment_id IS NULL`).

DAX — measures for a Power BI P&L

```
Total Sales     = SUM(Sales[Amount])
Total Cost      = SUM(Sales[Cost])
Gross Margin %  = DIVIDE([Total Sales] - [Total Cost], [Total Sales])   -- DIVIDE = safe /0

-- Filter context override: sales only for the North region, ignoring page slicers
North Sales     = CALCULATE([Total Sales], Sales[Region] = "North")

-- Row-by-row: value = qty * price computed per row, then summed
Revenue         = SUMX(Sales, Sales[Qty] * Sales[Price])

-- Time intelligence (needs a marked Date table)
Sales YTD       = TOTALYTD([Total Sales], 'Date'[Date], "31-03")   -- Indian FY year-end
Sales LY        = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
YoY %           = DIVIDE([Total Sales] - [Sales LY], [Sales LY])
```

`SUM` aggregates one column; `SUMX` iterates a table and evaluates an expression per row — use `SUMX` whenever the math is `A * B` at row level. `CALCULATE` is the one function to truly master: it *modifies filter context*.

pandas — reconciliation and reshaping

```
import pandas as pd

ledger = pd.read_excel("ledger.xlsx", sheet_name="GL")
bank   = pd.read_csv("bank_stmt.csv", parse_dates=["txn_date"])

# Clean + merge on a key
recon = ledger.merge(bank, on="utr_no", how="outer", indicator=True)
unmatched = recon[recon["_merge"] ≠ "both"]          # breaks to investigate

# Group & aggregate (the SQL GROUP BY equivalent)
summary = (ledger
           .groupby("cost_centre")
           .agg(total=("amount", "sum"), count=("amount", "size"))
           .reset_index())

# Pivot to a matrix (months across columns)
pt = ledger.pivot_table(index="account", columns="month",
                        values="amount", aggfunc="sum", fill_value=0)

ledger["amount"] = ledger["amount"].fillna(0)          # handle blanks
summary.to_excel("mis_summary.xlsx", index=False)      # ship it
```

Worked example / mini-project

Task: monthly MIS — revenue by region with a YoY view — from a raw sales table.

Sample `sales` table (₹ lakh):

sale_date	region	amount
2024-05-10	South	12.5
2024-05-22	North	8.0
2025-05-08	South	15.0
2025-05-19	North	9.5

Step 1 — SQL to pull the monthly base:

```
SELECT region,
       DATE_FORMAT(sale_date, '%Y-%m') AS mth,
       SUM(amount)                     AS rev
FROM   sales
GROUP  BY region, DATE_FORMAT(sale_date, '%Y-%m')
ORDER  BY region, mth;
```

Step 2 — load into Power BI, build measures:

```
Rev     = SUM(sales[amount])
Rev LY  = CALCULATE([Rev], SAMEPERIODLASTYEAR('Date'[sale_date]))
YoY %   = DIVIDE([Rev] - [Rev LY], [Rev LY])
```

For May: South ₹15.0L vs ₹12.5L LY → **+20%**; North ₹9.5L vs ₹8.0L → **+18.75%**. Drop a matrix visual (rows = region, values = Rev, Rev LY, YoY %) and a Date slicer.

Step 3 — same answer in pandas as a sanity check:

```
df["yr"] = df["sale_date"].dt.year
piv = df.pivot_table(index="region", columns="yr", values="amount", aggfunc="sum")
piv["yoy_%"] = (piv[2025] - piv[2024]) / piv[2024] * 100
print(piv.round(2))
```

Three tools, one number — that cross-check *is* the job.

How it's tested

- **SQL screen (30–45 min, HackerRank / live editor):** you're given 2–3 tables and asked "top 5 customers by revenue", "second-highest salary" (window function), or "invoices with no payment" (`LEFT JOIN … IS NULL`). Interviewers love `WHERE` vs `HAVING` and `INNER` vs `LEFT` .

- **Power BI / DAX case:** a take-home dataset — "build a P&L dashboard with YoY and a region slicer." They check whether your measures survive slicing (filter-context awareness) and whether you used `DIVIDE` not `/`.
- **pandas / Python round:** "read these two files, reconcile, output the breaks" or a `groupby` + `pivot_table` puzzle.
- **Verbal:** "Explain filter context." "SUM vs SUMX?" "How do you avoid a fan-out / duplicate join?"

Common mistakes & how pros avoid them

Mistake	Fix
Duplicated rows after a join (fan-out)	Join on a **unique** key; check row count before/after; aggregate before joining
`WHERE` used to filter an aggregate	Aggregates go in `HAVING`, not `WHERE`
Dividing by zero blows up DAX	Always `DIVIDE(a, b)` — returns blank on /0
Using `SUM` where math is row-level	Use `SUMX(table, qty*price)`
No Date table → time intelligence fails	Add a marked `Date` table, join it, use `TOTALYTD` / `SAMEPERIODLASTYEAR`
`merge` silently drops rows	Use `how="outer", indicator=True`, inspect `_merge`
Chained-assignment / SettingWithCopy warning	Use `.loc[]` or `.copy()`
Trusting NULL == NULL in SQL	Test with `IS NULL`; `COALESCE` before comparing

Learn-it roadmap & resources

Realistic time to interview-ready (2 hrs/day):

- **SQL:** 3–4 weeks. Free: SQLBolt, Mode Analytics SQL tutorial, LeetCode Database (Easy/Medium), StrataScratch.
- **DAX / Power BI:** 4–6 weeks. Free: Microsoft Learn "Power BI" path, SQLBI.com (Marco Russo), "Learn DAX" YouTube. Cert (optional but strong on a resume): **PL-300 Microsoft Power BI Data Analyst** (~US$165 / ~₹4,800).
- **pandas:** 3–4 weeks. Free: Kaggle "Pandas" + "Python" micro-courses, the official 10-minutes-to-pandas guide.

Practice on real data: your bank statement, a GST sales register, or public datasets (data.gov.in). Rebuild one MIS report end-to-end across all three tools — that single project teaches more than 20 tutorials.

Quick-reference

SQL

```
SELECT col, AGG(x) FROM t JOIN u ON t.k=u.k
WHERE ...  GROUP BY col  HAVING AGG(x)>n  ORDER BY col DESC  LIMIT n;
Joins: INNER (matched) | LEFT (keep left) | anti-join: LEFT + IS NULL
Window: SUM()/AVG() OVER(PARTITION BY .. ORDER BY ..), ROW_NUMBER(), RANK(), LAG()/LEAD()
NULLs: COALESCE(c,0) | IS NULL | NULLIF(a,b)
Dates: BETWEEN, DATEDIFF(), DATE_FORMAT()
```

DAX

```
SUM / AVERAGE / COUNTROWS(col)          -- aggregate one column
SUMX(tbl, expr)                         -- row-by-row then sum
CALCULATE(expr, filter1, filter2)       -- modify filter context (the key one)
DIVIDE(num, den[, 0])                   -- safe division
FILTER(tbl, cond) | ALL() | ALLEXCEPT() -- filter helpers
SAMEPERIODLASTYEAR / TOTALYTD / DATESYTD-- time intelligence
RELATED() | RELATEDTABLE()              -- cross-table lookup
```

pandas

```
pd.read_csv / read_excel / to_excel
df.merge(o, on="k", how="left"/"outer", indicator=True)
df.groupby("c").agg(t=("x","sum"), n=("x","size"))
df.pivot_table(index=, columns=, values=, aggfunc="sum", fill_value=0)
df.loc[mask, "col"] = v   |  df["c"].fillna(0)  |  df.drop_duplicates()
df.assign(newcol= ... )  |  df.sort_values("c", ascending=False)
```

Mental map: SQL `GROUP BY` = pandas `groupby` = DAX aggregate in filter context. SQL `JOIN` = pandas `merge` = DAX relationship. Same idea, three dialects.